

UNIVERSIDAD GALILEO
Postgrado en Diseño y Desarrollo de Software

INFRAESTRUCTURA EN LA NUBE
Proyecto de Curso

Sistema de Reservas y Disponibilidad de Parqueos

Entrega 1

INTEGRANTES

Ana Gabriela López Ortíz 18002988

Pablo Samuel Siquinajay Castro 22006477

Christian Javier Sosa Ruiz 22000102

Ciudad de Guatemala, 17 de mayo de 2026

ENTREGA 1

Resumen ejecutivo

Descripción del problema

Actualmente, muchas empresas cuentan con una cantidad limitada de espacios de parqueo en comparación con el número total de colaboradores. Con la implementación de modelos de trabajo híbrido (presencial y teletrabajo), la asistencia de empleados varía diariamente, lo que dificulta administrar de manera eficiente la disponibilidad de parqueos corporativos.

La falta de un sistema centralizado provoca conflictos de asignación, espacios desaprovechados y procesos manuales poco eficientes para gestionar reservas y disponibilidad.

Por ello, se propone desarrollar un Sistema de Reserva y Gestión de Parqueos Empresariales que permita a las empresas administrar de forma automatizada la disponibilidad de espacios para sus colaboradores. El sistema permitirá realizar reservas, visualizar disponibilidad en tiempo real, evitar conflictos de asignación y optimizar el uso de los parqueos disponibles, mejorando así la gestión interna y la experiencia de los colaboradores.

Actores

Actores Primarios

- **Usuario/Conductor:** Es la persona registrada en el sistema, la cual consulta disponibilidad, solicita y confirma reservas de espacios de parqueo.
- **Administrador:** Gestiona el inventario de espacios y consulta reportes de ocupación y asistencia, crea y edita características del parqueo.

Actores de Soporte

- **Servicio de autenticación:** Gestiona el registro e inicio de sesión de usuarios, controlando roles (Usuario o Administrador).
- **Servicio de almacenamiento en nube:** Persisten los comprobantes digitales generados tras cada reserva confirmada.
- **Servicio de mensajería asíncrona:** Ejecuta la liberación automática de espacios no confirmados tras 20 minutos y dispara las alertas de confirmación al usuario.
- **Servicio de notificaciones:** Envía alertas y confirmaciones de reserva al conductor (correo electrónico).

Casos de uso priorizados

1. Inicio de sesión

Como empleado, debo de acceder con un usuario previamente creado por el administrador e iniciar sesión para poder acceder al sistema de reservas.

Prioridad: Po

Criterio de éxito: El empleado inicia sesión con las credenciales asignadas por el administrador. El sistema emite un token de sesión válido con el rol correspondiente. Credenciales incorrectas retornan HTTP 401.

2. Consulta de disponibilidad en tiempo real

Como conductor, quiero consultar la disponibilidad de espacios filtrada por tipo de vehículo para saber si hay lugar antes de reservar.

Prioridad: Po

Criterio de éxito: El endpoint `GET /availability` retorna el estado de cada espacio (Disponible/Reservado/Ocupado) en tiempo real, respetando restricciones de rol del conductor.

3. Reserva de espacio

Como conductor, quiero reservar un espacio y recibir un comprobante digital para garantizar mi lugar.

Prioridad: Po

Criterio de éxito: El endpoint `POST /reservar` asigna el espacio atómicamente previniendo double-booking. El conductor recibe un comprobante QR en PDF por correo en menos de 30 segundos.

4. Gestor de usuarios para administrador

Como administrador, quiero gestionar los colaboradores registrados en el sistema, incluyendo crear, editar y desactivar cuentas y asignar roles. Controlar qué usuarios pueden reservar y qué tipo de espacios tienen permitido ocupar.

Prioridad: Po

Criterio de éxito: El administrador crea, edita y desactiva cuentas asignando roles (operativo, ejecutivo, visitante frecuente). Un usuario desactivado pierde acceso inmediatamente pero conserva su historial. Ningún colaborador puede modificar su propio rol ni el de terceros.

5. Gestor de parqueos para administrador

Como administrador, quiero configurar los espacios de parqueo disponibles, definiendo qué tipo de vehículo es permitido y qué roles de colaborador pueden ocuparlos, para que el sistema asigne reservas respetando las restricciones operativas de la empresa.

Prioridad: Po

Criterio de éxito: El administrador crea y edita espacios asignando identificador, tipo de vehículo y roles autorizados. Puede marcar espacios como inactivos por rango de fechas, quedando invisibles para reserva. Un colaborador con rol no autorizado no puede reservar un espacio restringido aunque esté disponible.

6. Liberación automática por no confirmación

Como sistema, quiero liberar automáticamente espacios no confirmados para optimizar la disponibilidad.

Prioridad: P1

Criterio de éxito: Si el conductor no confirma en 20 minutos, el espacio vuelve a Disponible de forma asíncrona y se notifica al conductor antes de la liberación.

7. Dashboard de ocupación para administrador

Como administrador, quiero visualizar un dashboard de ocupación para monitorear el estado del parqueo en tiempo real.

Prioridad: P2

Criterio de éxito: El dashboard presenta de forma gráfica el conteo de espacios en cada estado (Disponible, Reservado, Ocupado) actualizado en intervalos no mayores a 30 segundos. El administrador puede distinguir la ocupación por tipo de vehículo (moto/carro) sin necesidad de consultas adicionales.

8. Gestión de tarifas

Como administrador, quiero establecer precios diferenciados por tipo de vehículo.

Prioridad: P2

Criterio de éxito: El administrador define y actualiza tarifas para moto y carro. El cambio es inmediato y queda registrado con marca de tiempo para auditoría.

Funcionalidades específicas del proyecto:

1. Reserva con validación de placa registrada

El sistema solo permite reservar si la placa del vehículo está previamente registrada en el perfil del colaborador. No cualquier persona con cuenta puede reservar — tiene que haber un vehículo vinculado y aprobado. Un colaborador puede tener más de una placa (ej. carro y moto), pero solo puede tener una reserva activa por día.

2. Asignación de tipo de espacio por perfil de colaborador

El Admin puede categorizar colaboradores: ejecutivo, operativo, visitante frecuente. Cada categoría tiene acceso a un pool de espacios distinto. Un colaborador operativo no puede reservar un espacio marcado como ejecutivo aunque esté disponible. Esto refleja jerarquías reales que las empresas ya manejan manualmente hoy.

3. Historial de asistencia vehicular por colaborador

Dado que cada reserva está vinculada a un colaborador identificado, el sistema genera automáticamente un registro de qué días llegó en vehículo, a qué hora confirmó su llegada y cuánto tiempo ocupó el espacio. El admin puede exportar esto. Es un diferenciador fuerte porque las empresas hoy no tienen visibilidad de esto.

4. Liberación anticipada voluntaria

El colaborador puede liberar su espacio antes de la hora de salida desde la app (ej. salió temprano). Esto actualiza disponibilidad en tiempo real y permite que otro colaborador tome ese espacio el mismo día. Es distinto al timeout automático de 20 minutos — esta es una acción explícita del usuario que beneficia al resto.

5. Política de cancelación con penalización configurable

Si un colaborador cancela una reserva menos de N horas antes (configurable por el admin), el sistema lo registra como "cancelación tardía". Acumular X cancelaciones tardías en un mes puede bloquear temporalmente la capacidad de reservar. Esto desincentiva el comportamiento de reservar por precaución sin intención real de usar el espacio.

Mockups del FE

Se diseñaron 7 pantallas low-fi que cubren los casos de uso priorizados (P0, P1 y P2). Los wireframes completos se encuentran en el Anexo A.

Mapeo a conceptos del curso

Esta sección vincula las necesidades operativas de nuestro sistema de parqueos con el stack tecnológico del curso, vincula los componentes con las funcionalidades específicas que nuestro Sistema de Reservas y Disponibilidad de Parqueos va a hacer.

Componente del curso	Cómo lo ejercita su proyecto
Cómputo (API)	- Endpoint <code>GET/availability</code> consulta disponibilidad de parqueo filtrando por tipo de vehículo (moto, carro) del lado del cliente. - Endpoint <code>POST/reservar</code> ejecuta la lógica de validación de disponibilidad y asigna espacios en ese mismo instante.
Base de datos	Inventario de espacios con estados <i>Libre/Ocupado</i> y almacenamiento de datos de usuario registrados(perfil de datos) + sus placas vehiculares.

Almacenamiento de archivos	• Comprobantes digitales(en PDF/PNG) de finalización de reserva del parqueo. • Hosting de archivos estáticos; como imágenes de referencia de las entradas de los parqueos. Ayuda visual para el conductor.
Procesamiento asíncrono	• Si el usuario no confirma digitalmente su ingreso tras 20 minutos de la hora pactada, un <i>Worker</i> asíncrono procesa la cancelación, libera el cupo en la base de datos y lo marca el espacio de “Reservado” a “Disponible”.
Red	• El sistema expone únicamente el API Gateway a internet público vía HTTPS. Los servicios internos (base de datos, worker de liberación asíncrona) residen en subredes privadas sin acceso directo desde el exterior.
Seguridad	• Roles de IAM ; El "<i>Admin</i>" solo puede ver reportes y gestionar; el "<i>User</i>" solo genera reservas. • Cifrado de datos sensibles (ej. <code>license_plate_number_user</code> y <code>user_phone_number</code>).
Observabilidad	• <i>Alarma de saturación: notificación automática si la disponibilidad de espacios llega a cero</i>

Scope (in/out)

Declaración explícita de qué hará el sistema y qué queda fuera del diseño.

- **In:**
 - Gestión de identidad(Auth); Para el registro de usuarios al sistema. Mejora el control de usuarios que ingresan al sistema y los permisos que tiene(User/Admin)
 - Búsqueda de espacios disponibles; Muestra los espacios disponibles de parqueo de la empresa.
 - Gestión automática de reservas; Mediante una reserva de espacio en tiempo real que asegure atomicidad de espacio(ej. para que 2 o más personas no reserven el mismo espacio al mismo tiempo). El sistema rechaza a uno y rechaza al otro por orden de reserva.
 - Liberación asíncrona; Cancelación automática por no confirmación el día programado (asíncrono), con una espera máxima de 20 minutos y alertas de confirmación, entonces, liberar espacio antes ocupado.
 - Generación de comprobantes de confirmación de reservas; Generación de comprobante único digital e inalterable para el acceso. El sistema genera un QR en PDF que se guarda en el almacenamiento de la nube.
 - Dashboard interactivos de Disponibilidad/Ocupación;

- Un cliente podrá
 - Consultar disponibles(tendrá mensajes en la UI de “Ocupado”/”Disponible”),
 - Solicitar reserva, confirmar reserva.
 - Marcar en la app un botón de “Ocupar Parqueo”, y el estado en la BD cambia a "Ocupado"
- Para el Admin, podrá ver
 - En su inventario(mediante algún tipo de gráfico) los espacios disponibles, espacios reservados y actualmente ocupados.
 - Establecer el precio del parqueo por moto y carro.
 - Que usuarios han reservado y que días (brinda control de asistencia)
 - Respecto a la disponibilidad del parqueo, se usarán cantidad de parqueos disponibles(por motivos prácticos y sencillez) para ubicar qué número de parqueo es el que se encuentra como “Ocupado”/”Disponible”.
- **Out:**
 - Integración física con hardware.
 - Procesamiento de pagos reales; Por tema de complejidad, y costos.
 - **App Móvil Nativa**

Preguntas abiertas E1

¿Qué mecanismo concreto garantiza la atomicidad en la asignación de espacios? Se menciona que "el sistema rechaza a uno por orden de reserva", pero no se especifica si se usará locking optimista, pesimista, transacciones ACID, o algún mecanismo de cola (queue).

¿Cómo se maneja el race condition a nivel de base de datos cuando dos requests llegan en el mismo milisegundo?

¿Qué tecnología ejecuta la liberación automática a los 20 minutos? No se define si es un cron job, un mensaje en una cola (SQS, RabbitMQ), un scheduled function (Cloud Scheduler, Lambda EventBridge), o un timer en base de datos. Cada opción tiene implicaciones de confiabilidad distintas.

¿Cómo integraremos una función más eficaz cuando el cliente llegue “verdaderamente” al parqueo más que solo un clic de “Ocupar parqueo”? ¿Un IoT como sensor de parqueo sería suficiente?

Anexos

Anexo A: Mockups

Pantalla 1 - Registro e inicio de sesión: Esta pantalla cubre el acceso al sistema. Se presentan dos vistas: el formulario de inicio de sesión para usuarios existentes y el formulario de registro para nuevos usuarios.

The mockup shows two side-by-side white cards on a light beige background. The left card is for login, featuring a 'P' icon, the title 'ParkReserva', a login button, and fields for email and password. The right card is for registration, titled 'Crear cuenta', with fields for name, surname, email, vehicle type, license plate, and password, followed by a registration button.

ParkReserva
Iniciar sesión

Correo electrónico

Contraseña

Ingresar

¿No tienes cuenta? [Regístrate](#)

Crear cuenta
Completa tus datos

Nombre

Apellido

Correo electrónico

Tipo de vehículo
Carro

Placa

Contraseña

Crear cuenta

Pantalla 2 - Consulta de disponibilidad en tiempo real: Esta pantalla permite al conductor visualizar el estado actual de los espacios de parqueo. Se presenta una cuadrícula con cada espacio identificado por número, mostrando su estado mediante código de color: verde (Disponible), rojo (Ocupado) y amarillo (Reservado). El conductor puede filtrar por tipo de vehículo (moto/carro) antes de proceder a reservar.

The mockup displays a grid of parking spaces with their status (Available, Reserved, Occupied) and a summary of counts at the bottom. A filter for 'Carro' is selected.

Disponibilidad de espacios
Actualizado hace 3 segundos

Carro

Moto

■ Disponible

■ Reservado

■ Ocupado

A-01

A-02

A-03

A-04

A-05

B-01

B-02

B-03

B-04

B-05

5
Disponibles

2
Reservados

3
Ocupados

Reservar espacio

Pantalla 3 - Flujo de reserva: Guía al conductor a través del proceso de reserva de un espacio seleccionado. Se muestran los detalles del espacio elegido, los datos del vehículo registrado y la hora estimada de llegada. El sistema valida la disponibilidad de forma atómica al momento de confirmar, previniendo el double-booking. Al completarse, se genera el comprobante QR.

Confirmar reserva
Revisa los detalles antes de continuar

Espacio seleccionado

 **Espacio A-02**
Zona A - Nivel 1 Disponible

Datos del vehículo

Tipo: **Carro** Placa: **P-123ABC**

Hora estimada de llegada
08:30 AM 

El espacio se liberará automáticamente si no es confirmado en 20 minutos.

Cancelar **Confirmar reserva**

Pantalla 4 - Comprobante de reserva con código QR: Una vez confirmada la reserva, el sistema genera y presenta al conductor un comprobante digital único con código QR. Este comprobante queda disponible para descarga inmediata en formato PDF y es enviado al correo registrado del conductor. El código QR es inalterable y sirve como credencial de acceso al parqueo.



Reserva confirmada
Tu comprobante ha sido generado

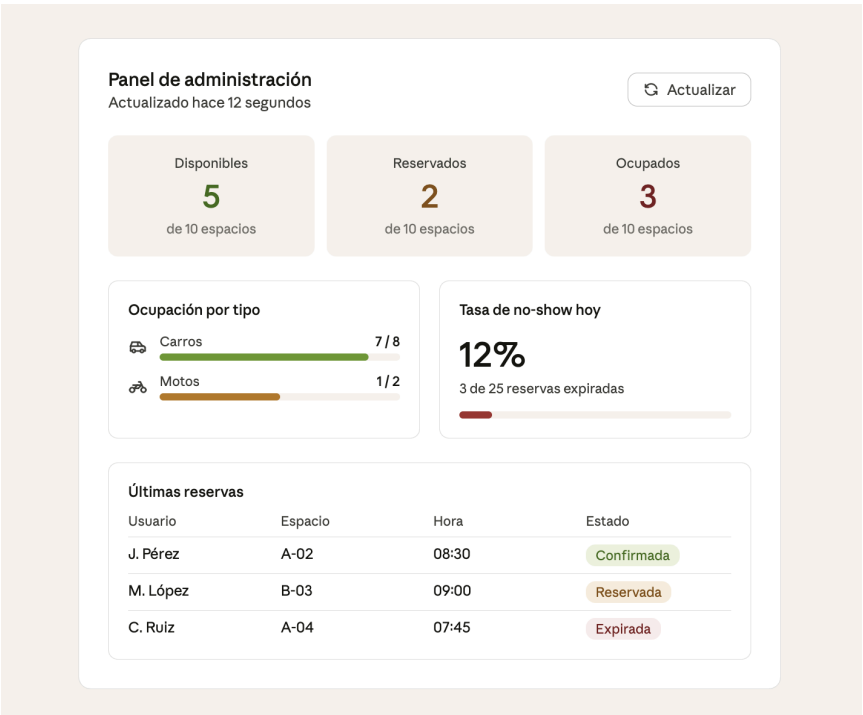
No. de reserva: **#RES-20260515-0042**
Espacio asignado: **A-02 - Zona A**
Placa: **P-123ABC**
Hora de llegada: **08:30 AM - 15 may 2026**



Presenta este QR al ingresar al parqueo

 **Descargar PDF**  **Enviar correo**

Pantalla 5 - Dashboard del administrador: Esta pantalla provee al administrador una visión general en tiempo real del estado del parqueo. Se presentan métricas de ocupación mediante gráficos y contadores por estado, diferenciados por tipo de vehículo.



Pantalla 6 - Gestión de precios por tipo de vehículo: Esta pantalla permite al administrador configurar y actualizar las tarifas diferenciadas por tipo de vehículo. Cada cambio queda registrado en el historial, para fines de auditoría.

Gestión de tarifas
Configura el precio por tipo de vehículo

Carro
Tarifa por hora (Q)
Q 10.00

Moto
Tarifa por hora (Q)
Q 5.00

Historial de cambios

Fecha	Tipo	Tarifa
15 may, 08:00	Carro	Q 10.00
10 may, 14:30	Moto	Q 5.00
01 may, 09:15	Carro	Q 8.00

Cancelar

Guardar cambios

Pantalla 7- Historial de reservas por usuario y fecha: Esta pantalla permite al administrador consultar y auditar el registro completo de reservas del sistema. La tabla es filtrable por usuario y rango de fechas, mostrando para cada entrada el espacio asignado, tipo de vehículo, estado final y marca de tiempo. Los registros son inmutables; ningún rol puede eliminarlos.

Historial de reservas
Registro inmutable de todas las reservas

Exportar

Usuario: Todos los usuarios
Fecha inicio: 01 may 2026
Fecha fin: 15 may 2026

Usuario	Espacio	Vehículo	Fecha / Hora	Estado
Juan Pérez	A-02	Carro	15 may · 08:30	Confirmada
María López	B-03	Moto	15 may · 09:00	Reservada
Carlos Ruiz	A-04	Carro	14 may · 07:45	Expirada
Ana González	B-01	Moto	13 may · 10:15	Cancelada
Juan Pérez	A-01	Carro	12 may · 08:00	Confirmada

Mostrando 5 de 42 registros

Anterior Siguiente

Anexo B — Anexo de IA

1. Prompt #1

"Actúa como un Arquitecto de Soluciones Cloud. Tengo un sistema de reserva de parqueos en Guatemala para un proyecto. Ayúdame a:

- 1. Redactar el cierre del Resumen Ejecutivo enfocándose en qué procesos se van a automatizar (evitar double-booking, liberación de espacios).*
- 2. Generar ejemplos específicos para una tabla que mapee las funcionalidades del sistema con los 7 componentes obligatorios del curso (Cómputo, DB, Almacenamiento, Red, Asíncrono, Seguridad, Observabilidad).*
- 3. Asegúrate de que los ejemplos no sean genéricos (no digas solo 'hacer login') sino que mencionen procesos reales de un parqueo inteligente."*

Nuestra reflexión

- **Qué aceptamos:** La estructura de la tabla y los ejemplos de procesos asíncronos, ya que es un "trade-off" interesante para el negocio.
- **Qué editamos:** Editamos ejemplos de procesos asíncronos, seguridad, red, observabilidad, Almacenamiento de archivos.
- **Qué descartamos:** Sugerencias de la IA sobre algunas funciones de red, como aislamiento de la base de datos y la lógica de negocio.

2. Prompt #2

"Actúa como Product Owner y genera una lista de Scope In/Out para un sistema de parqueos inteligente enfocado en un curso de Infraestructura Cloud, priorizando funcionalidades que usen bases de datos y colas de mensajes."

Reflexión del equipo

- **Qué aceptamos:** La estructura de "In/Out" sugerida por la IA, especialmente la idea de dejar fuera la integración física (sensores) y el desarrollo para una app móvil para no complicar el alcance técnico del curso.
- **Qué editamos:** Todas las sugerencias, ajustándose a un desarrollo más práctico y sencillo.
- **Qué descartamos:** La IA sugirió incluir "Machine Learning para predicción de demanda"; lo movimos a *Out-of-Scope* inmediatamente para mantener el proyecto ejecutable en 3 meses.

ENTREGA 2

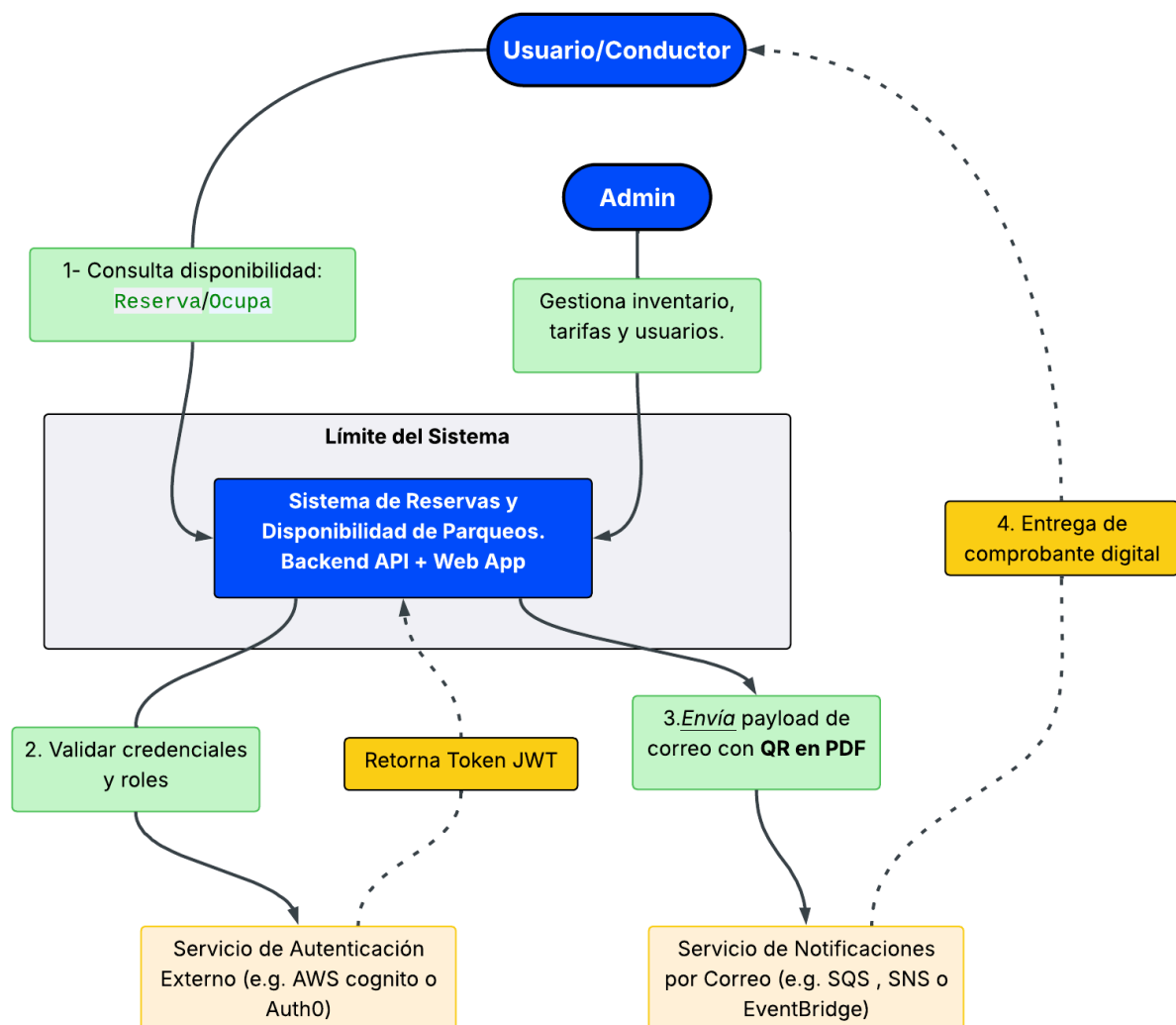
Resumen de cambios

- **Definición de la Estrategia de Concurrencia (Atomicidad):**

Se resolvió la duda de la *Entrega 1* sobre el mecanismo para evitar el *double-booking*. Se descartó la necesidad de implementar colas de mensajes bloqueantes o bloqueos pesimistas a nivel de aplicación; en su lugar, se optó por delegar la atomicidad a la base de datos mediante **índices únicos parciales** en PostgreSQL, simplificando la capa de cómputo.

Diagrama de contexto

El siguiente diagrama delimita las fronteras del Sistema de Reserva y Gestión de Parques Empresariales, también, definiendo los actores externos y sistemas de terceros.



Descripción de las Entidades del Diagrama

A. Límite del Sistema (System Boundary)

- **Sistema de Reservas y Disponibilidad de Parques:** Comprende todo el software desarrollado por el equipo (interfaces web de usuario/administrador y el backend API de funciones). Se encarga de procesar la lógica de negocio, calcular la disponibilidad en tiempo real, derivar estados vehiculares y gobernar la persistencia de datos relacionales y punteros de archivos.

B. Actores Externos (Usuarios Primarios)

- **Usuario/Conductor:** Interactúa con el sistema para consultar la disponibilidad de espacios en tiempo real, enviar solicitudes de reserva, registrar voluntariamente la ocupación o liberación anticipada y recibir sus comprobantes digitales.
- **Administrador (Personal de operaciones de la empresa):** Interactúa con el sistema para dar de alta o baja cuentas de colaboradores, configurar restricciones vehiculares por perfil(solo moto o carro) y auditar el historial de asistencia vehicular de la corporación.

C. Sistemas Externos de Terceros (Sistemas de Soporte)

- **Servicio de Autenticación Externo (e.g., AWS Cognito / Auth0):** Sistema gestionado externo encargado de validar las credenciales de acceso al sistema. Le quita la responsabilidad al sistema de almacenar contraseñas, retornando tokens criptográficos (JWT) que contienen el rol (`system_role`) del usuario para la posterior autorización en el API.
- **Servicio de Notificaciones (e.g., AWS SQS/SNS/Event Bridge):** Plataforma externa de entrega de correos electrónicos. Recibe las instrucciones del sistema transaccional de parques junto con el archivo PDF/PNG del comprobante generado, y se encarga de enviarlo directamente al buzón del conductor final.

Decisión de cómputo

Servicio elegido: AWS ECS Fargate

El backend del sistema se desplegará como un contenedor Docker gestionado por ECS Fargate. Fargate es la modalidad serverless de contenedores en AWS: el equipo define la task definition (imagen, CPU, memoria, puertos) y AWS gestiona el servidor, el sistema operativo y los parches. No se administra ninguna instancia EC2 directamente.

Por qué Fargate sobre las alternativas

AWS Lambda fue descartado porque el sistema expone una API REST con múltiples endpoints que necesitan estar disponibles de forma continua. Lambda está diseñado para ejecuciones cortas y puntuales y no es el modelo adecuado para un servidor HTTP

persistente con conexiones a base de datos que debe responder en tiempo real a consultas de disponibilidad cada 30 segundos.

EC2 con Auto Scaling fue descartado porque requiere administrar el servidor subyacente: actualizaciones del sistema operativo, parches de seguridad, configuración del runtime y gestión del ciclo de vida de las instancias. Para un equipo de tres personas en un proyecto de curso, ese overhead operativo no agrega valor al diseño del sistema.

Trade-offs

1. Disponibilidad continua vs. costo en baja demanda

Fargate mantiene el contenedor corriendo aunque no haya tráfico, lo que genera costo incluso en horarios de baja actividad (noches, fines de semana). Lambda cobraría solo por invocación y resultaría más económico para un sistema con uso muy intermitente. Para este sistema, donde el dashboard del administrador se refresca cada 30 segundos durante el horario laboral y las reservas ocurren en ventanas predecibles de la mañana, la disponibilidad continua está justificada: el costo de una respuesta lenta o un cold start en el momento pico de reservas (07:00–09:00) supera el ahorro nocturno.

2. Sin servidor que administrar vs. menor granularidad de control.

Al delegar la gestión del servidor a AWS, el equipo gana velocidad operativa pero pierde control sobre la configuración del sistema operativo, la red a nivel de host y la capacidad de instalar dependencias del sistema directamente. Para el backend de este proyecto, una API REST con dependencias definidas y empaquetadas en un contenedor, ese nivel de control no es necesario. Si en el futuro el sistema requiriera acceso a hardware especializado o configuraciones de red avanzadas a nivel de instancia, EC2 sería la opción correcta.

Desventaja reconocida

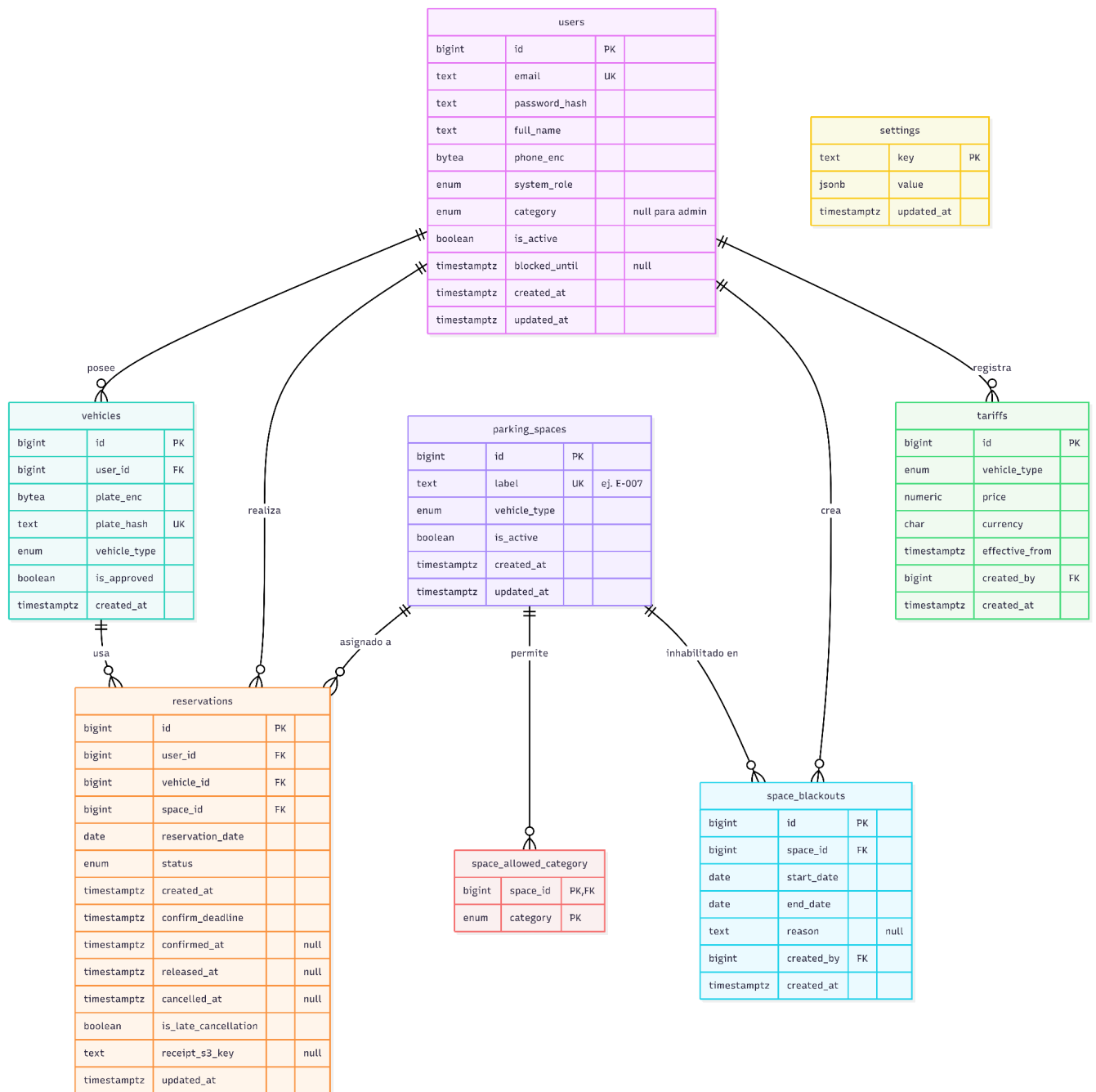
Fargate tiene una latencia de arranque de 20 a 30 segundos cuando escala desde cero tareas. Si el servicio se detiene por inactividad y recibe una solicitud en ese instante, la primera respuesta se verá afectada. Para el MVP se acepta este riesgo manteniendo al menos una tarea corriendo en todo momento, lo que elimina el cold start a costa de no escalar a cero. En producción, esto se mitigaría con una política de Auto Scaling que nunca baje de una tarea mínima.

Modelo de datos

El dominio se modela como un esquema relacional sobre PostgreSQL 16 (RDS), en modo single-company (un despliegue para una empresa, sin tabla de tenants). Se eligió un modelo relacional porque el negocio gira en torno a invariantes transaccionales (no permitir doble reserva, una reserva activa por día) y la integridad referencial entre colaboradores, vehículos, espacios y reservas, estas son propiedades que una base relacional garantiza de forma nativa.

Entidades principales

Entidad	Rol en el dominio
users	Colaboradores y administradores en una sola tabla, diferenciados por <code>system_role</code> .
vehicles	Vehículos del colaborador (1:N). Reservar exige una placa registrada y aprobada.
parking_spaces	Inventario de espacios; cada uno con un tipo de vehículo admitido.
space_allowed_category	Relación M:N que define qué categorías de colaborador pueden usar cada espacio.
space_blackouts	Inhabilitación de un espacio por rango de fechas.
reservations	Núcleo del modelo: sostiene disponibilidad, atomicidad, liberación, asistencia y el comprobante.
tariffs	Historial de precios append-only (auditoría); el precio vigente es la fila más reciente por tipo.
settings	Política configurable por el admin (ventana de cancelación, máximo de cancelaciones tardías).



Tres principios sostienen la coherencia del modelo

1. El estado del espacio (**Disponible/Reservado/Ocupado**) se *deriva*, no se **almacena**. Se calcula por (**espacio, fecha**) a partir de la reserva activa. Una columna **estado** fija no puede expresar "ocupado hoy, libre mañana" y se desincronizaría con las reservas.
2. **Atomicidad por índice único parcial**, no por bloqueos de aplicación (ver patrón de acceso 2).
3. **Dos ejes de "rol" separados**: **system_role** (admin/driver, para permisos) y **category** (ejecutivo/operativo/visitante_frecuente, para el pool de espacios accesible). No se fusionan.

Patrones de acceso principales

#	Patrón de acceso	Tipo	Frecuencia	Estructura que lo soporta
1	Consultar disponibilidad (GET /availability), filtrada por tipo de vehículo y categoría del conductor	Lectura	Muy alta	LEFT JOIN parking_spaces ↔ reservations del día activo; índice (reservation_date, status); exclusión de inactivos y space_blackouts.
2	Crear reserva (POST /reservar), atómica	Escritura (alta contención)	Alta	Índices únicos parciales (space_id, reservation_date) y (user_id, reservation_date) con WHERE status IN ('reservada','ocupada').
3	Confirmar llegada / liberar (ocupar, salida, liberación anticipada)	Escritura	Media	UPDATE reservations de status + marcas de tiempo (confirmed_at, released_at).
4	Liberación automática a los 20 min (worker P1)	Lectura + escritura batch	Periódica	Índice (status, confirm_deadline) para escanear reservas por expirar.
5	Historial de asistencia por colaborador (export del admin)	Lectura	Baja	Índice (user_id, reservation_date); campos confirmed_at/released_at.
6	Conteo de cancelaciones tardías del mes (penalización Fn5)	Lectura	Baja	is_late_cancellation + índice (user_id, reservation_date).
7	Precio vigente / historial de tarifas	Lectura	Baja	tariffs append-only + vista current_tariffs (DISTINCT ON (vehicle_type)).
8	Gestión (CRUD) de usuarios y espacios	Escritura	Baja	Claves primarias e índices únicos (email, label, plate_hash).

El patrón #1 es el más frecuente (la pantalla de disponibilidad se refresca cada ≤ 30 s) y por eso se optimiza con un índice dedicado y un conjunto de datos pequeño (decenas de espacios). El patrón #2 es el más crítico en correctitud: la **atomicidad la garantiza la base de datos**. Dos reservas simultáneas para el mismo espacio/día ejecutan **INSERT** concurrentes; el índice único parcial deja pasar solo uno y el otro recibe una violación de unicidad que el API traduce a **HTTP 409**. Es ACID, sin bloqueo pesimista ni cola de mensajes, responde la pregunta de E1 sobre cómo se evita el double-booking.

Qué va en base de datos vs. almacenamiento de objetos

Dato	Dónde	Justificación
Usuarios, vehículos, espacios, reservas, tarifas, settings	PostgreSQL (RDS)	Datos estructurados y relacionales, registros pequeños, requieren transacciones (reserva atómica), integridad referencial (FKs) y consultas indexadas (disponibilidad, historial).
Comprobantes de reserva (QR en PDF/PNG)	S3 (object storage)	Blobs binarios, inmutables (write-once), de tamaño medio, que se sirven por descarga/URL prefirmada. No requieren consultas relacionales; almacenarlos en BD inflaría el tamaño y degradaría backups.
Imágenes estáticas de referencia (entradas del parqueo)	S3	Contenido estático servido directamente; no es dato transaccional.
Puntero al comprobante (<code>reservations.receipt_s3_key</code>)	PostgreSQL	La BD guarda solo la clave/URL del objeto en S3, no el binario. Mantiene la fila ligera y enlaza el dato relacional con el objeto.

Decisión para MVP: no se introduce una capa de caché dedicada (ej. Redis/ElastiCache). La consulta de disponibilidad (patrón #1) opera sobre un conjunto pequeño (decenas de espacios) con un índice apropiado, por lo que PostgreSQL cumple holgadamente los SLAs (< 2 s de respuesta, reflejo del cambio en ≤ 30 s) sin caché. Añadir una caché ahora introduciría un problema de **invalidación y consistencia** (datos desactualizados) que contradice el requisito de tiempo real, a cambio de un beneficio de rendimiento que aún no se necesita.

Preguntas abiertas E2

1. ¿Mantener enums para **vehicle_type**/categorías o migrar a tablas de catálogo si el negocio necesita agregarlos sin desplegar cambios de esquema?
2. ¿Cuándo conviene **particionar reservations por fecha** al crecer el historial, y qué política de retención/archivado aplica?
3. Dado que la base de datos PostgreSQL (RDS) y el backend se alojarán en subredes privadas (acorde a E1 en *Componentes del Curso*), ¿cuál será el mecanismo de red óptimo para que nuestro cómputo se comunique de forma segura con Amazon S3 para subir los PDFs de reservas?
 - a. ¿Utilizaremos un *Gateway VPC Endpoint* para evitar que el tráfico viaje por la internet pública o abriremos rutas de NAT Gateway?
4. Para cuando **Disponibilidad = 0** ¿Es más eficiente calcular el estado en tiempo real con *Lambda* periódica que publique una métrica customizada en *CloudWatch*, o deberíamos emitir **eventos** directamente desde la BD en cada mutación de reserva?

Anexo IA

1. Lo que le pedimos a la IA

- *"Actúa como un arquitecto de datos en la nube. Ayúdanos a estructurar una estrategia de persistencia para un sistema de reservas de parqueos corporativos de tamaño mediano (decenas de espacios). Compáranos el uso de un enfoque puramente relacional (PostgreSQL) contra uno NoSQL (DynamoDB), considerando la regla de negocio crítica: evitar double-booking y garantizar una sola reserva por día por usuario."*

2. Análisis y Reflexión del Equipo

- **Qué aceptamos:** En primera instancia la IA nos recomendó 100% una Base de datos relacional (ej. PostgreSQL). Evaluamos nuestra lógica de negocio, como por ejemplo *"permitir solo una reserva activa por usuario por día"* o *"validar si el espacio pertenece a la categoría asignada al colaborador"*. Estas requieren un aseguramiento en la integridad y consistencia en datos, además de manejar consistencia en datos, algo que con los principios ACID de una BD relacional nos queda excelente.
- **Qué editamos:**
- **Qué descartamos:** La IA sugirió implementar una capa de almacenamiento en caché en memoria distribuida usando *Redis (Amazon ElastiCache)* para acelerar el endpoint de disponibilidad de espacios (**GET /availability**), bajo el pretexto de que la lectura se actualiza cada 30 segundos. El equipo descartó la propuesta de inmediato para el MVP, debido a que el volumen de datos de un parqueo mediano no justifica la complejidad técnica extra de controlar la invalidación de la caché, pudiendo resolverse eficientemente con índices nativos en PostgreSQL sin generar

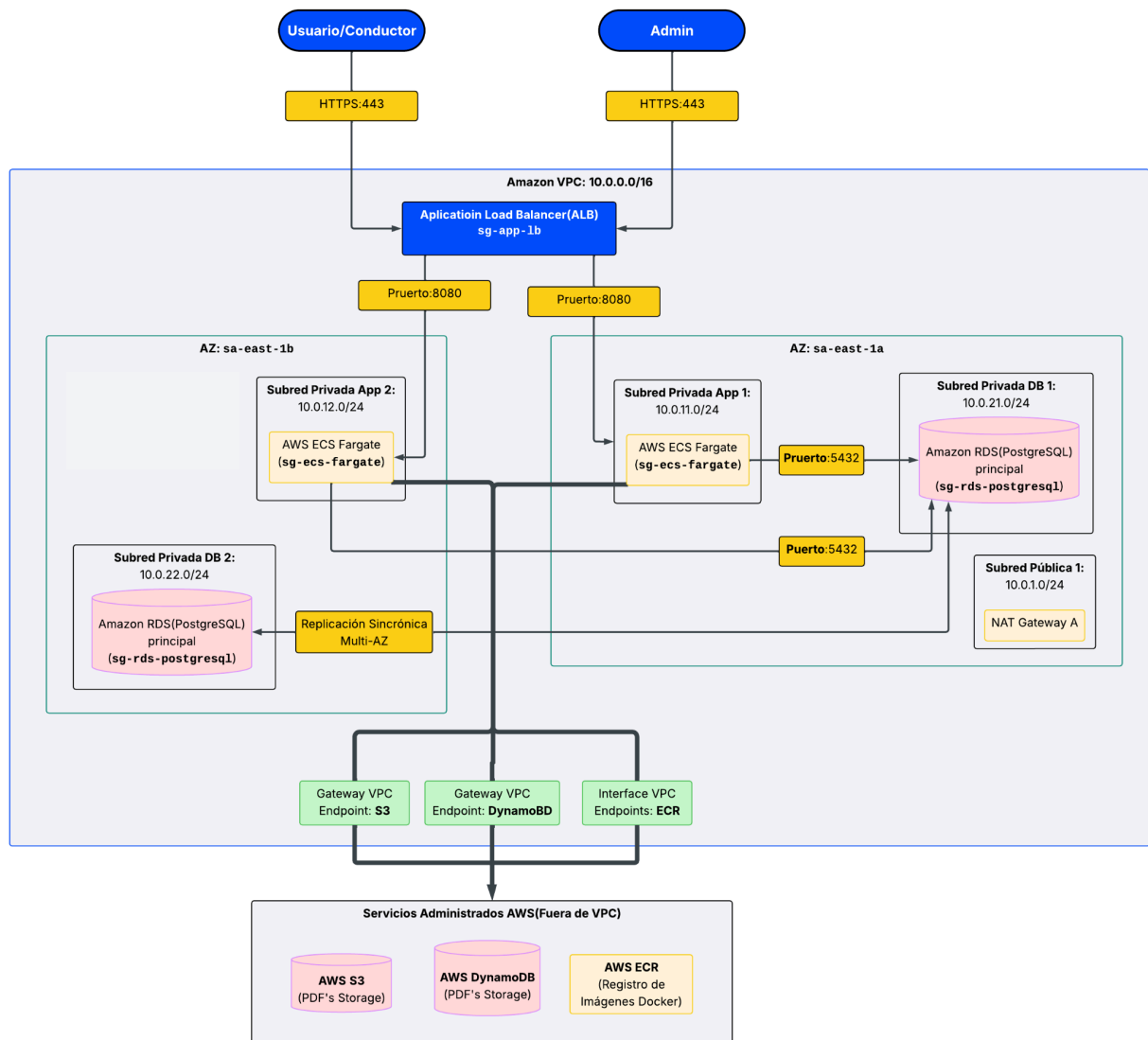
inconsistencias de datos en tiempo real. Añadir una caché ahora introduciría un problema de **invalidación y consistencia** (datos desactualizados) que **contradice el requisito de tiempo real**, a cambio de un beneficio de rendimiento que aún no se necesita.

ENTREGA 3

Resumen de cambios

- Nueva subsección: Decisión NAT Gateway vs. VPC Endpoints con análisis específico al sistema
- Nueva sección: Decisiones de Alcance LATAM (centralización VPC, AZs, región parametrizable)
- Adopción de Amazon ECR debido a ECS Fargate: Las tareas de ECS Fargate requieren comunicarse con Amazon ECR para descargar las imágenes de los contenedores de forma segura, ya sea a través del NAT Gateway o mediante un Interface VPC Endpoint dedicado (AWS PrivateLink).
- **Cambio arquitectónico:** región base us-west-2 → sa-east-1 como default LATAM. AZ naming actualizado. Es la única región AWS dentro del territorio latinoamericano, lo que la convierte en el argumento de residencia de datos más sólido frente a clientes corporativos LATAM que exigen que sus datos no salgan del continente.

Diagrama de contenedores (primera versión)



Diseño de red

Para garantizar la alta disponibilidad, la tolerancia a fallos y la seguridad del **Sistema de Reserva y Gestión de Parques Empresariales**, se ha diseñado una topología de red aislada utilizando una **Amazon VPC (Virtual Private Cloud)**. Este diseño mitiga los riesgos de exposición de datos sensibles de los colaboradores (como nombres, IDs y números de placas vehiculares) y asegura la resiliencia del sistema durante las horas pico de alta concurrencia (6:00 AM - 9:00 AM).

1. Direccionamiento IP y Segmentación de Subredes (Subnetting)

Se ha asignado a la VPC el bloque de direccionamiento principal **10.0.0.0/16**, lo que proporciona un espacio teórico de hasta 65,536 direcciones IP privadas, garantizando escalabilidad para futuras expansiones físicas del parqueo.

1.1 Justificación del Bloque 10.0.0.0/16 para Contexto LATAM

¿Por qué /16 y no un bloque más pequeño como /20 o /22?

La decisión no responde al número de colaboradores de la empresa cliente (que puede ser de 300 a 5,000 empleados para un parqueo formal en LATAM), sino a 2 criterios de ingeniería:

a) Dimensionamiento por servicios de infraestructura, no por usuarios

Las IPs de la VPC las consumen los componentes de infraestructura, no los usuarios finales del sistema. El inventario real por despliegue es el siguiente:

Componente	IPs consumidas	aproximadas
Tareas ECS Fargate activas (2–8 tasks en horario pico)		≤ 8
Instancias RDS PostgreSQL (primaria + réplica)		2
Nodos del ALB (gestionados por AWS)		2–6
NAT Gateway (1 por AZ)		1–2
Total operativo actual		≤ 18 IPs

Con subredes /24 (251 IPs utilizables cada una), el diseño actual ocupa 6 subredes × 251 = 1,506 IPs asignadas en el espacio de 65,536 (ocupación del 2.3%). Ese headroom no es desperdicio: es capacidad reservada para expansiones previsibles sin re-numeración.

b) Suficiencia para empresas LATAM con parqueos formales

Para una empresa con hasta 3,000 colaboradores con acceso al sistema de parqueos y un parqueo con 500 espacios, el sistema jamás necesitará más de ~50 IPs de infraestructura activas simultáneamente. El /16 es suficiente, y lo seguirá siendo incluso si el sistema expande a 10 empresas cliente en el mismo despliegue.

Para cumplir con las prácticas de **Alta Disponibilidad (HA)** sin triplicar los costos de transferencia de datos inter-zona (*Cross-AZ Data Transfer*), la infraestructura se distribuye simétricamente en **2 Zonas de Disponibilidad (Availability Zones: sa-east-1a y sa-east-1b)**. La red se segmenta verticalmente en una arquitectura de tres capas (*3-Tier Architecture*).

A. Capa Pública (DMZ, Demilitarized Zone)

Diseñada como barrera de seguridad entre la red interna de una organización y el Internet público.

- **Subred Pública 1 (AZ sa-east-1a):** 10.0.1.0/24 (251 IPs utilizables). Hospeda el nodo 'A' del Application Load Balancer (ALB) y el NAT Gateway de la zona 'a'.
- **Subred Pública 2 (AZ sa-east-1b):** 10.0.2.0/24 (251 IPs utilizables). Hospeda el nodo 'B' del ALB y provee redundancia de entrada pública.

B. Capa Privada de Cómputo (Application Layer)

Aislada del acceso directo a internet. Aquí reside la lógica de negocio del sistema.

- **Subred Privada App 1 (AZ sa-east-1a):** 10.0.11.0/24 (251 IPs utilizables). Hospeda los contenedores del API corriendo sobre **AWS ECS Fargate**.
- **Subred Privada App 2 (AZ sa-east-1b):** 10.0.12.0/24 (251 IPs utilizables). Hospeda las réplicas de los contenedores de ECS Fargate para balancear la carga activa.

C. Capa Privada de Datos (Data Layer)

Máximo nivel de aislamiento dentro de la VPC. No tiene rutas de salida hacia el internet.

- **Subred Privada Datos 1 (AZ sa-east-1a):** 10.0.21.0/24 (251 IPs utilizables). Hospeda la instancia primaria de **Amazon RDS PostgreSQL**.
- **Subred Privada Datos 2 (AZ sa-east-1b):** 10.0.22.0/24 (251 IPs utilizables). Hospeda la instancia secundaria/réplica de lectura de RDS PostgreSQL, configurada con despliegue Multi-AZ para conmutación por error (*failover*) automatizada.

2. Enrutamiento e Interconexión de Servicios Cloud

2.1 Flujo de Entrada (Inbound Routing):

1. El usuario (conductor) o el administrador realiza una petición HTTP/HTTPS al dominio del sistema.
2. La solicitud llega al **Internet Gateway (IGW)** asociado a la VPC, el cual la deriva al **Application Load Balancer (ALB)** distribuido en las subredes públicas.
3. El ALB actúa como proxy inverso: termina la conexión SSL/TLS y distribuye el tráfico balanceado hacia las tareas de **AWS ECS Fargate** en las subredes privadas de cómputo a través del puerto interno 8080.

2.2 Flujo de Salida y Optimización de Costos (Outbound Routing):

- Decisión

Los contenedores de ECS Fargate requieren salida controlada a internet para tres operaciones críticas del sistema: (1) validación de tokens contra Autho, (2) despacho de correos con comprobante PDF a través de SendGrid/SES, y (3) descarga de imágenes de contenedores desde Amazon ECR durante el arranque o escalado. Adicionalmente, el sistema sube comprobantes en PDF/QR a Amazon S3 y escribe registros de auditoría en DynamoDB.

La arquitectura implementa una **estrategia híbrida** con dos rutas de tráfico diferenciadas:

- **Ruta A — NAT Gateway** (0.0.0.0/0 en tablas de enrutamiento privadas): Para tráfico con destino a servicios de terceros externos (Autho, SendGrid) donde no existe alternativa de VPC Endpoint.
- **Ruta B — Gateway VPC Endpoints** (S3 y DynamoDB): Para tráfico con destino a servicios internos de AWS, que se desvía completamente del NAT Gateway y viaja por la red troncal privada de AWS.

- Trade-offs

- **Trade-off 1: NAT Gateway único (AZ-a) vs. NAT Gateway redundante (una instancia por AZ)**

El diseño actual despliega un único NAT Gateway en `sa-east-1a`. Las tareas ECS en `sa-east-1b` enrutan su salida hacia ese NAT en zona cruzada.

Dimensión		Opción actual: NAT único en AZ-a	Alternativa: NAT por AZ
Costo	fijo mensual	~\$32.4/mes (1 gateway × \$0.045/hr × 720h)	~\$64.8/mes (2 gateways)
Resiliencia de salida		Si <code>sa-east-1a</code> falla, los contenedores en <code>sa-east-1b</code> pierden capacidad de llamar a Autho y SendGrid	Cada AZ opera de forma autónoma; la caída de una zona no afecta la salida de la otra

Impacto en el sistema de parqueos	Durante el horario pico 06:00–09:00, un fallo de <code>sa-east-1a</code> impide que ECS en <code>sa-east-1b</code> valide tokens: el sistema queda parcialmente inoperante aunque la base de datos siga activa	Mantiene la disponibilidad completa del flujo de reservas aunque caiga una AZ
Decisión para el proyecto	Aceptado para MVP; el Cross-AZ data transfer (\$0.01/GB) en tráfico de notificaciones es mínimo (~15 MB/día estimado). En producción con SLA formal, se añade el segundo NAT.	Recomendado cuando el sistema tenga SLA contractual con empresas cliente

La postura actual refleja que el MVP opera sin SLA contractual. Al formalizar el servicio con clientes LATAM, el costo adicional de ~\$32/mes se justifica por mantener el flujo de autenticación y notificaciones resiliente ante fallas de AZ, que según AWS tienen una frecuencia histórica de ~2–4 eventos de degradación por año por AZ.

- **Trade-off 2: Gateway VPC Endpoints (S3 + DynamoDB) vs. enrutar por NAT Gateway**

Dimensión	Gateway VPC Endpoint	Ruta por NAT
Costo de datos	\$0.00 por GB transferido	\$0.045/GB procesado por el NAT
Latencia de upload (S3)	Tráfico en red interna de AWS; evita saltos de internet (~30–80ms de diferencia en round-trip)	El PDF del comprobante (~150 KB) transita por internet antes de volver a AWS
Superficie de ataque	Los datos de comprobantes (que vinculan nombre, placa e ID de reserva) nunca salen de la red de AWS	Cualquier ruta por internet amplía el perímetro de inspección
Complejidad operativa	Requiere gestionar entradas adicionales en tablas de enrutamiento (2 entradas prefix-list por endpoint)	Sin configuración adicional
Portabilidad Terraform	Se declara como recurso <code>aws_vpc_endpoint</code> en el módulo de red; exportado como output para consumo del módulo de cómputo (principio Sesión 6: módulos consumen outputs, nunca IDs literales)	No aplica

3. Estrategia de Firewalls: Grupos de Seguridad (Security Groups)

Se aplica de forma estricta el principio de mínimo privilegio mediante reglas encadenadas por referencia de ID de Security Group (no por rangos CIDR), siguiendo el siguiente principio:

"Las reglas deben referenciar objetos de Security Group río arriba, no rangos CIDR, para que las reglas sigan al objeto y no a la IP."

Esta práctica es especialmente crítica en entornos ECS Fargate donde las IPs de los contenedores cambian dinámicamente en cada tarea.

1. **sg-application-alb (Asociado al ALB):**
 - **Ingreso (Inbound):** Permite puertos 80 (HTTP) y 443 (HTTPS) desde cualquier origen (0.0.0.0/0).
 - **Egreso (Outbound):** Permite tráfico únicamente hacia el puerto 8080 apuntando explícitamente al identificador sg-ecs-fargate.
2. **sg-ecs-fargate (Asociado a los contenedores de la API):**
 - **Ingreso (Inbound):** Permite tráfico en el puerto 8080 **únicamente** si la fuente proviene del grupo sg-application-lb. Bloquea cualquier petición directa de internet.
 - **Egreso (Outbound):** Permite tráfico total (0.0.0.0/0) para conectarse con los endpoints externos de autenticación, el NAT Gateway y los VPC Endpoints.
3. **sg-rds-postgresql (Asociado a la Base de Datos):**
 - **Ingreso (Inbound):** Permite conexiones en el puerto 5432 **únicamente** si provienen del grupo de seguridad de cómputo sg-ecs-fargate. Se deniega el acceso incluso a las subredes públicas.
 - **Egreso (Outbound):** Denegado por defecto (No requiere iniciar conexiones salientes).

Preguntas abiertas

- ¿Qué pasaría si expandimos las zonas de disponibilidad? ¿Qué pasaría si nuestro sistema se expande y llega a continentes europeos o asiáticos? ¿Qué zonas habríamos de disponer y cómo manejamos los costos?
- Dado que el sistema está centralizado en una única VPC en la región **sa-east-1** (São Paulo) para dar servicio a múltiples países de LATAM, ¿cómo afectaría la latencia de red acumulada y un eventual corte de fibra submarina internacional al SLA del Sistema (< 30 segundos)? ¿Vale la pena diseñar un esquema de replicación asincrónica de la base de datos RDS hacia una región secundaria con costos de transferencia transcontinental reducidos (ej. **us-east-1**), o el riesgo se mitiga delegando la resiliencia puramente a las zonas de disponibilidad locales (**sa-east-1a** y **sa-east-1b**)?

- Al procesarse miles de reservas diarias por cada sede corporativa, ¿en qué momento exacto del tiempo es financieramente óptimo realizar la transición de los archivos desde *S3 Standard* hacia *S3 Glacier Flexible Retrieval* o *Deep Archive*?
- Considerando que nuestro público objetivo se de LATAM, también encuentra en México y Centroamérica, el diseño actual centralizado en EE. UU. introduce una latencia base debido a la distancia geográfica. ¿Es justificable mitigar esta latencia agregando una tercera Zona de Disponibilidad local para absorber ráfagas de concurrencia y caídas de datacenters, o el problema se debe atacar implementando una capa perimetral como *Amazon CloudFront* o *AWS Global Accelerator*? ¿Cómo balanceamos el costo fijo de añadir NAT Gateways y transferencia de datos inter-AZ adicionales frente al costo variable por transferencia de datos saliente (*Data Transfer Out*) que cobra una red de distribución de contenido (CDN)?

Anexo IA

1. Qué se le pidió a la IA

Se le solicitó una estructura de subredes con máscaras CIDR optimizadas para una arquitectura de tres capas en AWS utilizando ECS Fargate y RDS, buscando alta disponibilidad en 2 AZs convenientes para países de LATAM.

2. Análisis y Reflexión del Equipo

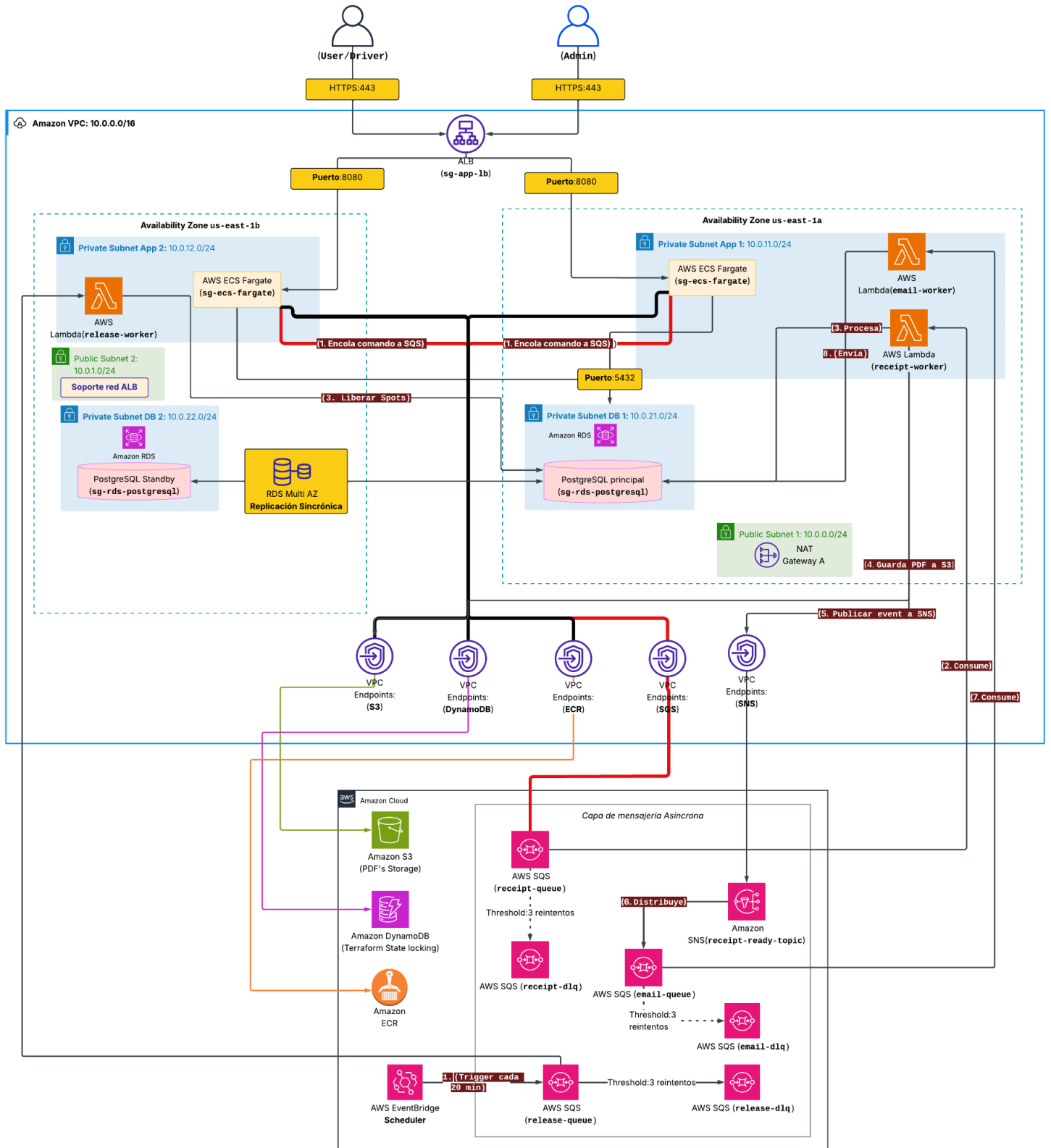
- **Qué se aceptó:** El direccionamiento propuesto basado en bloques `/24` para cada subred, ya que provee un balance adecuado entre el aislamiento de entornos (251 IPs utilizables por zona) y evita el desperdicio excesivo de direccionamiento en la VPC principal `/16`.
 - Se aceptó el manejo de las subredes en `sa-east-1` para garantizar menor latencia entre la mayoría de países de LATAM, aunque si queremos expandirnos más, podríamos aceptar uno o más en EU para asegurar mejor latencia en países como MX y centroamérica; claro que implican más costos y hay mejores opciones como un CDN.
- **Qué se editó:** La IA sugería colocar las tablas de enrutamiento públicas de manera unificada. El equipo editó la propuesta separando las tablas de enrutamiento públicas y privadas de manera independiente por cada Zona de Disponibilidad, garantizando que el fallo de un NAT Gateway en la AZ `sa-east-1a` no bote la conectividad saliente de la AZ `sa-east-1b`.
- **Qué se descartó:** La recomendación automatizada de utilizar un *AWS Transit Gateway* y *VPC Peering* para conectar los microservicios. Se descartó inmediatamente porque el alcance transaccional actual del sistema de parqueos se resuelve de forma centralizada en una única VPC monolítica de red, evitando costos innecesarios de ruteo inter-VPC.
 - Además descartamos las residencias principales en estados unidos(donde se encuentran las mayorías de datacenters).

ENTREGA 4

Resumen de cambios

- La generación del comprobante y la liberación automática de reservas se sacaron del contenedor del API y ahora corren como workers asíncronos vía SQS y SNS, porque antes, si el contenedor se reiniciaba, el trabajo se perdía sin reintento.
- Se agregaron a la arquitectura las colas SQS con sus Dead Letter Queues, el tópico SNS, los workers en Lambda y el disparador EventBridge Scheduler para la liberación periódica.
- La API se mantiene en Fargate y solo los workers asíncronos se implementan en Lambda, por ser procesos cortos disparados por eventos.
- Se aclaró que las colas no intervienen en la doble reserva: la atomicidad sigue garantizada por los índices únicos parciales en PostgreSQL de la Entrega 2.
- El control de idempotencia se resuelve con una tabla en PostgreSQL en lugar de Redis, en coherencia con la Entrega 2 que ya había descartado ElastiCache por el bajo volumen.
- Se prevén VPC Endpoints (PrivateLink) para SQS y SNS en las subredes privadas, alineado con los endpoints que la Entrega 3 ya usaba para S3 y DynamoDB.
- **EventBridge Scheduler como disparador periódico:** Se incorporó Amazon EventBridge Scheduler para ejecutar periódicamente el barrido de reservas expiradas, reemplazando el timer interno dentro del contenedor Fargate que se perdía al reiniciar la tarea.
- Se cambiaron las regiones de `sa-east-1` a `us-east-1` dado que si bien es cierto nuestro público objetivo es LATAM, es muy posible que personas del extranjero utilicen (por ejemplo E.U.) la app ya que no hay motivos de restricciones (de nuestro sistema) en cuanto a países con la disponibilidad de usar el sistema. Aunque si no fuera así, se seguiría manteniendo el sistema con pocas regiones y usando simplemente CDN ya que nuestra clase de sistema no es tan dinámico y tan personalizado.

Diagrama de contenedores actualizado



Resumen de cambios

- Migración de región: Se actualizó la región base de sa-east-1 a us-east-1 en todas las AZs, VPC Endpoints y nomenclatura de recursos, según el lineamiento del proyecto.
- **Adición de VPC Endpoints para SQS y SNS (PrivateLink):** Se incorporaron Interface VPC Endpoints (com.amazonaws.us-east-1.sqs, com.amazonaws.us-east-1.sns) en las subredes privadas, siguiendo el mismo patrón de la Entrega 3 para ECR. Esto asegura que el tráfico de mensajería asíncrona nunca salga a internet y no incurra en costos de NAT Gateway.
- **Workers asíncronos en AWS Lambda (VPC-enabled):** Se trasladaron los procesos de generación de comprobantes, liberación de reservas y envío de correos a funciones Lambda en lugar de ejecutarse dentro del contenedor Fargate. Lambda es el modelo correcto para procesos cortos y orientados a eventos, mientras que Fargate continúa alojando la API REST persistente (consistente con la decisión de la Entrega 2). Las funciones Lambda se configuran con VPC-attach para acceder a RDS desde las subredes privadas.
- **Arquitectura de colas SQS con Dead Letter Queues:** Se diseñaron tres colas principales (receipt-queue, release-queue, receipt-email-queue) con sus respectivas DLQ, cada una configurada con maxReceiveCount = 3 y backoff exponencial. Los mensajes fallidos se conservan íntegros en las DLQ para auditoría y reinyección manual.
- **Tópico SNS con fan-out hacia SQS:** El evento ReceiptReadyEvent se publica en un tópico SNS que suscribe a receipt-email-queue (cola SQS), en lugar de suscribir Lambda directamente. Esto hereda el mecanismo de reintentos y DLQ de SQS para el envío de correos.
- Se justifica la integración de **Amazon EventBridge Scheduler** como el componente encargado de disparar de forma periódica (cada N minutos) el proceso de liberación de espacios de parqueo expirados. Su uso mitiga el antipatrón de mantener temporizadores en memoria dentro de los contenedores efímeros de ECS Fargate (Express.js), los cuales generarían ejecuciones duplicadas al escalar horizontalmente o pérdida de tareas programadas ante reinicios de los contenedores. EventBridge actúa de forma serverless y desacoplada, garantizando un mecanismo confiable de alta disponibilidad para la consistencia del inventario de parqueos sin incurrir en costos de cómputo ocioso.

Flujos asíncronos

1. Distinción entre Comando y Evento

El diseño asíncrono del sistema se apoya en dos tipos de mensaje, y conviene definirlos con claridad porque se usan en situaciones distintas.

Un comando es una instrucción para que algo se ejecute. Describe una acción pendiente, tiene un único destinatario y se espera que se procese una sola vez. Los comandos viajan por Amazon SQS, que garantiza la entrega del mensaje y permite reintentarlo si el procesamiento falla.

Un evento es la notificación de algo que ya ocurrió. No le ordena nada a nadie en particular, solo anuncia un cambio de estado para que los interesados reaccionen por su cuenta. Un mismo evento puede tener varios destinatarios. Los eventos viajan por Amazon SNS, que distribuye una copia del mensaje a cada suscriptor (patrón fan-out).

La diferencia se resume así: el comando se nombra en imperativo y representa trabajo por hacer; el evento se nombra en pasado y representa trabajo ya realizado.

Cómputo de los workers

La API REST seguirá ejecutándose sobre ECS Fargate, como se decidió en la Entrega 2, porque es un servidor HTTP persistente que debe responder de forma continua. Los workers asíncronos, en cambio, se ejecutarán como funciones AWS Lambda: son procesos cortos, disparados por la llegada de un mensaje, que no requieren un servidor corriendo de forma permanente.

Los tres flujos asíncronos del sistema

Del análisis del backend actual en Express.js surgen tres procesos que conviene volver asíncronos:

- a) **GenerateReceiptCommand (comando):** Cuando **POST /reservar** confirma la reserva y responde HTTP 201, en lugar de generar el comprobante en el mismo proceso, el API publica este comando en una cola SQS. Un worker lo consume, genera el PDF con código QR, lo sube a Amazon S3 y actualiza el campo **receipt_s3_key** de la reserva. Es una relación uno a uno: un comando produce un comprobante.
- b) **ReceiptReadyEvent (evento):** Cuando el comprobante queda guardado en S3, el worker publica este evento en un tópico SNS para anunciar que la reserva está lista. Varios suscriptores reaccionan de forma independiente: el envío del correo con el comprobante al conductor (requisito del caso de uso 3: comprobante por correo en menos de 30 segundos) y, como extensión futura, la actualización del dashboard de ocupación del administrador. Es una relación uno a muchos.
- c) **ReleaseExpiredReservationCommand (comando):** La liberación automática de reservas no confirmadas a los 20 minutos deja de ejecutarse como una tarea programada dentro del contenedor del API y pasa a dispararse con Amazon EventBridge Scheduler, que coloca periódicamente este comando en SQS para que un worker libere las reservas vencidas.

Productores y consumidores

Mensaje	Transporte	Productor	Consumidor
GenerateReceiptCommand	Amazon SQS	API Express.js	Worker de comprobantes
ReceiptReadyEvent	Amazon SNS	Worker de comprobantes	Servicio de correo y dashboard

ReleaseExpiredReservationCommand	EventBridge Scheduler → SQS	EventBridge Scheduler	Worker de liberación
----------------------------------	-----------------------------	-----------------------	----------------------

2. Definición de Payloads

Cada mensaje transporta un objeto JSON con dos partes: un encabezado de control común a todos los mensajes y un bloque **data** con la información de negocio específica. Esta separación permite que los workers procesen el control del mensaje (identidad, tipo, momento de emisión) sin necesidad de interpretar el contenido de negocio.

Campos de control comunes

Los tres mensajes incluyen los siguientes campos en el nivel superior:

Campo	Tipo	Propósito
event_type	string	Nombre del mensaje (GenerateReceiptCommand, ReceiptReadyEvent, ReleaseExpiredReservationCommand). Permite distinguir y enrutar el mensaje.
idempotency_key	string (UUID)	Identificador único del mensaje. Es la base del control de duplicados descrito en la sección de idempotencia.
timestamp	string (ISO 8601)	Momento de emisión del mensaje. Se usa para auditoría, ordenamiento y para medir la latencia frente al SLA de 30 segundos del comprobante.

Decisión de seguridad

Los mensajes transportan únicamente identificadores (**reservation_id**, **user_id**, **space_id**). Datos personales como el nombre del conductor o el número de placa no se incluyen en el payload. El worker que necesita esa información la obtiene directamente de la base de datos: lee **users.full_name** y descifra **vehicles.plate_enc**, igual que lo hace el backend actual. De este modo, ningún dato sensible circula por las colas ni por los tópicos, manteniendo la coherencia con la política de cifrado de datos definida desde la Entrega 1. Como consecuencia de diseño, el worker de comprobantes requiere conexión a la base de datos RDS.

Payload 1: GenerateReceiptCommand

Lo produce el endpoint **POST /reservar** tras confirmar la reserva; lo consume el worker de comprobantes.

Los campos del bloque data corresponden a los valores que el backend ya maneja en la respuesta 201 de **POST /reservar**: **reservation_id** es el identificador generado por el

INSERT, y `space_id`, `vehicle_id` y `reservation_date` provienen de la misma respuesta. Con estos identificadores el worker recupera de la base de datos el nombre del conductor y la placa, genera el PDF con código QR, lo sube a Amazon S3 y actualiza `reservations.receipt_s3_key`.

```
{
  "event_type": "GenerateReceiptCommand",
  "idempotency_key": "a3f1c8e2-5b7d-4e9a-8c1f-2d6b4a9e0f3c",
  "timestamp": "2026-05-17T08:30:00Z",
  "data": {
    "reservation_id": 4042,
    "user_id": 87,
    "space_id": 12,
    "vehicle_id": 5,
    "reservation_date": "2026-05-17"
  }
}
```

Payload 2: ReceiptReadyEvent

Lo produce el worker de comprobantes cuando el PDF queda almacenado en S3; lo consumen el servicio de correo y, como extensión futura, el dashboard del administrador.

El campo `receipt_s3_key` sigue el formato que el backend ya utiliza (`receipts/<id>.pdf`). El suscriptor de correo usa `user_id` para localizar la dirección del conductor y `receipt_s3_key` para generar una URL prefirmada y adjuntar el comprobante. El evento se mantiene ligero de forma intencional, solo anuncia que el comprobante de una reserva está disponible, y cada suscriptor decide cómo reaccionar.

```
{
  "event_type": "ReceiptReadyEvent",
  "idempotency_key": "b7e2d9f4-1a3c-4f8b-9d2e-6c5a8b1f0e4d",
  "timestamp": "2026-05-17T08:30:04Z",
  "data": {
    "reservation_id": 4042,
    "user_id": 87,
    "receipt_s3_key": "receipts/4042.pdf",
  }
}
```

Payload 3: ReleaseExpiredReservationCommand

Lo produce Amazon EventBridge Scheduler de forma periódica; lo consume el worker de liberación.

A diferencia de los otros dos mensajes, este no opera sobre una reserva específica, sino que dispara un barrido. El campo `scan_before` indica al worker el instante de corte: debe liberar toda reserva en estado reservada cuyo `confirm_deadline` sea anterior a ese momento, cambiándolas al estado expirada. Este comportamiento corresponde directamente a la consulta que el worker actual ya ejecuta sobre el índice.

```
{
  "event_type": "ReleaseExpiredReservationCommand",
  "idempotency_key": "c1d4e7a0-9f2b-4c6d-8e3a-1b5f7d9c2a8e",
  "timestamp": "2026-05-17T08:50:00Z",
  "data": {
    "scan_before": "2026-05-17T08:50:00Z"
  }
}
```

3. Estrategia de Idempotencia

En un sistema de mensajería asíncrona, un mismo mensaje puede llegar más de una vez. Amazon SQS garantiza entrega al menos una vez (at-least-once), lo que significa que, ante reintentos o demoras en el procesamiento, la cola puede entregar un duplicado. La idempotencia es la propiedad que asegura que procesar el mismo mensaje dos veces produzca el mismo resultado que procesarlo una sola vez. Cada worker debe protegerse contra duplicados.

El sistema aplica tres estrategias, una por flujo, todas resueltas en la base de datos sin introducir servicios adicionales.

a) Flujo 1: GenerateReceiptCommand

Antes de generar el comprobante, el worker consulta si la reserva ya tiene poblado el campo `receipt_s3_key`. Si ya contiene una clave, significa que un mensaje anterior generó el comprobante, por lo que el worker descarta el duplicado sin realizar trabajo. Si el campo está vacío (NULL), procede a generar el PDF, subirlo a S3 y poblar el campo. Como refuerzo, el archivo en S3 se nombra con una clave determinista (`receipts/<id>.pdf`). Aun si dos ejecuciones llegaran a coincidir, ambas escribirían el mismo objeto con contenido idéntico, sin generar comprobantes duplicados.

b) Flujo 2: ReceiptReadyEvent

Este es el único flujo que requiere un mecanismo explícito, porque el envío de un correo no es idempotente por sí mismo: enviarlo dos veces haría que el conductor

reciba dos correos. Para evitarlo, el suscriptor de correo mantiene una tabla de control en PostgreSQL (**processed_messages**) que usa la **idempotency_key** como llave primaria. Antes de enviar el correo, el suscriptor intenta insertar la clave del mensaje; si la inserción falla por violación de unicidad, el mensaje ya fue procesado y el envío se omite. Este mecanismo reutiliza el mismo patrón de índice único que el sistema ya emplea para garantizar la atomicidad de las reservas, aplicando una técnica que el proyecto ya domina en lugar de introducir una nueva.

c) Flujo 3: **ReleaseExpiredReservationCommand**

Este flujo es idempotente por la naturaleza de su operación. El worker ejecuta una actualización condicionada al estado actual de la reserva:

```
UPDATE reservations
SET status = 'expirada'
WHERE status = 'reservada' AND confirm_deadline <
:scan_before
```

Si el mensaje se procesara dos veces, la segunda ejecución no encontraría filas que cumplan la condición, ya que las reservas afectadas dejaron de estar en estado reservada. La cláusula **WHERE status = 'reservada'** actúa como el mecanismo de idempotencia: reprocesar el mensaje no produce ningún efecto adicional.

PostgreSQL en lugar de un servicio de caché dedicado

El control de idempotencia se resuelve en la base de datos PostgreSQL existente y no en un servicio de caché en memoria como Amazon ElastiCache (Redis). Esta decisión es coherente con la Entrega 2, donde se descartó introducir Redis para el MVP por considerar que el volumen del sistema —decenas de espacios, reservas concentradas en ventanas predecibles— no justifica la complejidad operativa de un servicio adicional. El volumen de mensajes asíncronos es igualmente bajo, por lo que una tabla de control en la base de datos es suficiente y evita añadir un componente nuevo que administrar.

4. Manejo de Fallos y DLQ

El backend actual maneja los fallos de la peor manera posible para un proceso crítico: cuando la generación del comprobante falla, el error solo se registra en el log y se descarta, sin reintento ni recuperación. Lo mismo ocurre en el worker de liberación. El comprobante simplemente nunca se crea y nadie se entera. Trasladar estos procesos a SQS resuelve precisamente este problema: convierte un fallo que se pierde en un log en un fallo que se reintenta automáticamente y que, de no poder resolverse, se preserva para auditoría.

Reintentos con backoff exponencial

Cuando un worker no logra procesar un mensaje ya sea por una falla de generación del PDF, una indisponibilidad momentánea de S3 o de la base de datos, SQS no descarta el mensaje, sino que lo vuelve a poner disponible para reintentarlo. Entre un intento y el siguiente, el tiempo de espera crece de forma exponencial (backoff exponencial), implementado mediante

el incremento progresivo del visibility timeout del mensaje. Esto evita saturar un servicio que está temporalmente caído y le da tiempo a recuperarse.

Threshold de reintentos

Cada cola principal se configura con un máximo de 3 intentos de procesamiento, definido por el parámetro **maxReceiveCount** en la política de redrive de SQS. Tres intentos son suficientes para absorber fallos transitorios (un timeout de red, un reinicio breve de la base de datos) sin insistir indefinidamente en un mensaje que es irrecuperable. Una vez superado ese umbral, el mensaje se traslada automáticamente a su Dead Letter Queue.

Dead Letter Queue por flujo

Cada cola principal tiene su propia Dead Letter Queue (DLQ), lo que permite diagnosticar cada flujo de forma independiente. Los nombres siguen la convención de recursos del proyecto (<nombre>-<entorno>-<recurso>):

- El flujo de comprobantes utiliza la cola <nombre>-<entorno>-receipt-queue, respaldada por su cola de mensajes fallidos <nombre>-<entorno>-receipt-dlq. Un mensaje que llega a esta DLQ indica que el comprobante no pudo generarse tras tres intentos, de modo que la reserva existe pero quedó sin comprobante y requiere intervención manual.
- El flujo de liberación utiliza la cola <nombre>-<entorno>-release-queue y su correspondiente <nombre>-<entorno>-release-dlq. Un mensaje en esta DLQ indica que el barrido de liberación falló de forma persistente, lo que significa que las reservas vencidas no llegaron a liberarse y el equipo debe revisarlas.
- Finalmente, el envío de correo utiliza la cola <nombre>-<entorno>-receipt-email-queue y su <nombre>-<entorno>-receipt-email-dlq. Un mensaje en esta DLQ indica que el correo con el comprobante no pudo entregarse tras tres intentos.

Los mensajes que llegan a una DLQ no se procesan automáticamente: quedan almacenados íntegros para inspección y auditoría manual. El equipo puede examinarlos, identificar la causa raíz del fallo, corregirla y, si corresponde, reinyectarlos a la cola principal (operación de redrive). Este es exactamente el comportamiento que hoy no existe: en lugar de un error perdido en un log, el mensaje fallido se conserva completo.

Manejo de fallos en el evento SNS

A diferencia de SQS, Amazon SNS no ofrece por sí mismo un mecanismo de reintentos con DLQ equivalente. Por ello, el suscriptor de correo, que necesita garantías de entrega, no se suscribe directamente al tópico SNS, sino que lo hace a través de su propia cola SQS (<nombre>-<entorno>-receipt-email-queue). De este modo, el envío del correo hereda el mismo mecanismo de reintentos y DLQ que el resto de los flujos: el tópico **ReceiptReadyEvent** publica en esa cola, y el worker de correo la consume con la misma política de 3 intentos y backoff exponencial.

Relación con la idempotencia

El mecanismo de reintentos hace que un mensaje pueda procesarse más de una vez, lo que vuelve indispensable la estrategia de idempotencia descrita en la sección anterior. Ambos mecanismos trabajan en conjunto: los reintentos garantizan que el trabajo llegue a completarse, y la idempotencia garantiza que reprocesar un mensaje no produzca efectos duplicados.

Preguntas abiertas

1. ¿Qué métricas y alarmas mínimas necesitamos sobre las colas para operar con confianza? En particular, ¿conviene alarmar sobre la antigüedad del mensaje más viejo (**ApproximateAgeOfOldestMessage**) en la cola de comprobantes para detectar que el SLA de 30 segundos se está incumpliendo, o basta con alarmar cuando lleguen mensajes a las DLQ? ¿Cuál es el umbral razonable para cada una?
2. Definimos que los datos personales no viajan en el payload y que los workers leen de RDS, pero queda por detallar el acceso de red: ¿los workers Lambda se colocan dentro de la VPC para alcanzar RDS y los VPC Endpoints de SQS/SNS, asumiendo el costo y la latencia de arranque que eso implica, o existe una alternativa? ¿Qué política IAM de mínimo privilegio aplica a cada worker (qué cola puede leer, en qué tópico puede publicar)?
3. La tabla **processed_messages** en PostgreSQL crece de forma indefinida. ¿Qué política de retención o limpieza aplicamos para que no degrade la base de datos con el tiempo, y a partir de qué volumen de mensajes la validación de idempotencia en PostgreSQL dejaría de ser suficiente frente a una solución como Redis con expiración por TTL?

Anexo IA

1. Lo que le pedimos a la IA

Durante el diseño de los flujos asíncronos consultamos a la IA en tres frentes:

Sobre la distinción comando/evento

Actúa como arquitecto de soluciones en AWS. Tengo un backend Express.js que hoy genera un comprobante PDF de forma síncrona tras crear una reserva, y un worker de liberación que corre como cron dentro del mismo contenedor. Quiero pasar esto a un patrón asíncrono. Ayúdame a distinguir qué partes deben modelarse como comando (SQS) y cuáles como evento (SNS), y por qué.

Sobre la idempotencia

En un sistema de mensajería con SQS que garantiza entrega at-least-once, ¿cómo garantizo idempotencia en los workers? Compárame validar las claves contra Redis (ElastiCache) frente a usar la base de datos relacional que ya tengo (PostgreSQL), para un sistema de bajo volumen.

Sobre el manejo de fallos

Para mis colas SQS, ¿qué threshold de reintentos y qué política de Dead Letter Queue es razonable? Mi notificación va por SNS: ¿cómo le doy reintentos y DLQ a un suscriptor de SNS?

2. Análisis y Reflexión del Equipo

Durante el diseño de los flujos asíncronos se tuvo la siguiente reflexión:

Sobre la distinción comando/evento

- **Qué aceptamos:** La distinción comando/evento y modelar el comprobante como comando y la notificación como evento.
- **Qué editamos:** Los nombres de los mensajes, para que coincidieran con identificadores ya existentes en el código.
- **Qué descartamos:** Encolar la reserva para evitar doble reserva, por contradecir la atomicidad por índices únicos de la Entrega 2.

Sobre la idempotencia

- **Qué aceptamos:** El concepto de clave de idempotencia y que un UPDATE condicionado al estado es idempotente por naturaleza.
- **Qué editamos:** Aplicamos una estrategia distinta por flujo en lugar de una sola técnica uniforme.
- **Qué descartamos:** Usar Redis (ElastiCache) para las claves, por coherencia con la Entrega 2; queda como evolución futura.

Sobre el manejo de fallos

- **Qué aceptamos:** El threshold de 3 reintentos con backoff exponencial y el patrón SNS→SQS para el suscriptor de correo.
- **Qué editamos:** Una DLQ independiente por flujo, alineada a la convención de nombres de nuestro Terraform.

ENTREGA 5

Resumen de cambios

- **Plan de observabilidad definido:** En la Entrega 4 el sistema no contaba con ninguna estrategia formal de visibilidad operativa. Se documenta el estado actual del logging estructurado (Pino v9 + pino-http v10) ya presente en el backend y se identifican sus brechas críticas: ausencia de identificador de correlación por petición, falta de `userId` en los logs y ningún mecanismo de métricas, trazado distribuido ni dashboards.
- **Correlación de peticiones con `requestId`:** Se diseña el mecanismo para cerrar la brecha de correlación más crítica, añadiendo un campo `requestId` único generado con `crypto.randomUUID()` en el middleware de pino-http, heredado por todos los logs del ciclo de vida de cada solicitud. Una vez que `requireAuth` verifica el JWT, el `userId` se incorpora al mismo logger hijo.
- **Métricas RED por endpoint principal:** Se definen las métricas de Rate, Errors y Duration para los endpoints críticos del sistema (`POST /reservar`, `GET /availability`) con sus fuentes concretas en CloudWatch Logs Insights, métricas nativas del ALB, RDS y SQS.
- **Estrategia de alertas mínimas para producción:** Se definen cuatro alarmas en CloudWatch con sus umbrales, ventanas de evaluación y justificación de impacto de negocio: tasa de errores 5xx elevada, mensajes en cualquier Dead Letter Queue, espacio libre en RDS por debajo del 15%, y número de tareas ECS del API en cero.
- **Secrets manager ahora con VPC Endpoint:** En esta entrega se agrega un nuevo VPC Endpoint de tipo Interface para Secrets Manager (`com.amazonaws.us-east-1.secretsmanager`) y que se asocia al mismo `sg-parking-vpce`. Esto mantiene todo el tráfico interno a AWS sin pasar por NAT.
- **Nueva Lambda administrada por Secret Managers:** La rotación automática de `parking/prod/rds-credentials` en la sección 2.2 de esta entrega introduce una Lambda administrada por Secrets Manager que no forma parte del cómputo de aplicación y no requiere SG ni IAM role propio del equipo (AWS la gestiona), pero sí necesita que el SG de RDS permita conexiones desde la subred donde Secrets Manager despliega esa Lambda.
- **Se formalizó el enum `reservation_status`:** Se establecieron cinco valores (reservada, ocupada, liberada, expirada, cancelada) La E2 había definido las columnas de marca de tiempo pero no enumeró explícitamente los estados; la E5 los consolida a partir del modelo de datos real. No es un cambio de diseño, sino la formalización de lo que las columnas ya implicaban.

- **Se adoptó el enum de tipos de vehículo:** Se adoptó el uso de auto / moto / camioneta, reemplazando la nomenclatura "moto/carro" usada en E1 y E2, por coherencia con el modelo de datos implementado.
- **Se documentó el API surface tal como está implementado en el código:** La creación de reservas es POST /reservar y las transiciones del ciclo de vida son POST /reservations/:id/confirm, /release y /cancel. Se descartó renombrarlas a un estilo REST genérico para mantener fidelidad con el backend existente.

Detalle del componente más complejo

El ciclo de vida de la reserva como state machine

El componente más complejo del sistema no es un endpoint ni un servicio aislado, sino el ciclo de vida de una reserva: la máquina de estados que gobierna la tabla *reservations*. Se eligió como el área a detallar porque es el único punto donde convergen todas las decisiones difíciles tomadas a lo largo del proyecto: la atomicidad por índice único parcial (Entrega 2), el estado derivado en lugar de almacenado (Entrega 2), la liberación automática por timeout (Entrega 1/Entrega 4), la liberación anticipada voluntaria (Entrega 1), la generación asíncrona del comprobante (Entrega 4) y la política de cancelación con penalización (Entrega 1). Un ingeniero que entienda esta máquina de estados entiende el motor del sistema.

Notas de diseño

☐ Estado activo (borde sólido)

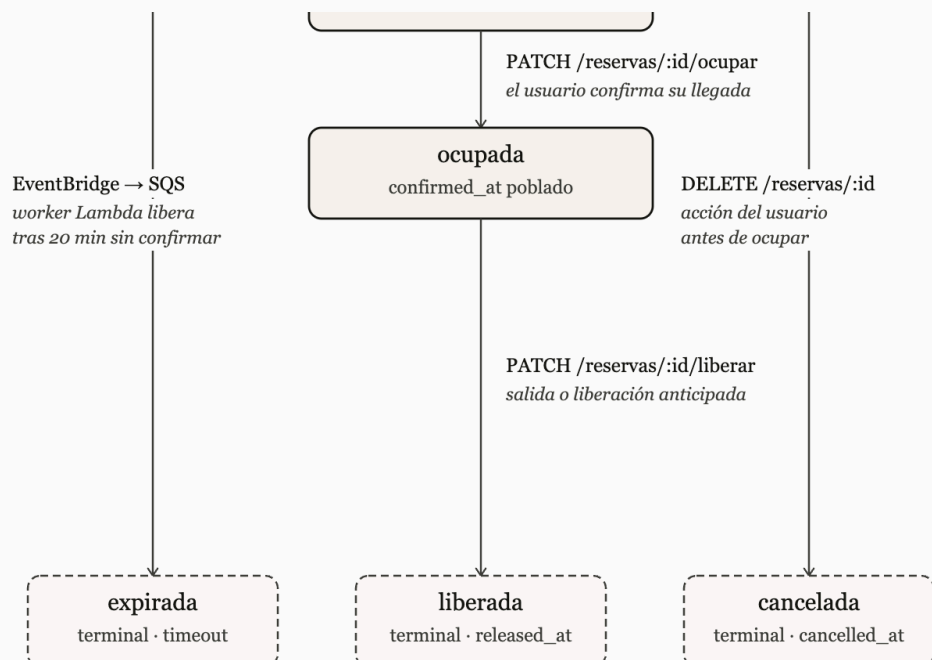
☐ Estado terminal (borde punteado)

Toda transición es un UPDATE condicionado al estado de origen: es idempotente y segura ante concurrencia.

Solo la creación (POST /reservar) requiere el índice único parcial que previene el double-booking.

Al entrar a «reservada» el API encola GenerateReceiptCommand (SQS): el comprobante es efecto lateral, no una transición.

En cualquier estado terminal el espacio vuelve a derivarse como disponible para esa fecha.



Conviene precisar una distinción que el documento ha usado de forma implícita desde la E1 y que aquí se formaliza: existen dos conceptos de "estado" distintos que no deben confundirse.

- El estado del espacio (Disponible / Reservado / Ocupado) es lo que ve el usuario en la pantalla de disponibilidad. Este estado se deriva, no se almacena (principio 1 de la Entrega 2), se calcula por (*space_id*, *reservation_date*) a partir de la reserva activa de ese día.
- El estado de la reserva (*reservations.status*) es lo que sí se persiste como columna. Es el sujeto de esta máquina de estados.

La relación entre ambos es directa: una reserva en estado reservada hace que su espacio se derive como "Reservado"; una en estado ocupada lo deriva como "Ocupado"; y cualquier estado terminal (liberada, expirada, cancelada) hace que el espacio vuelva a derivarse como "Disponible" para ese día.

Los cinco estados de *reservations.status*

El conjunto de estados consolida las columnas que la Entrega 2 ya definió (*confirmed_at*, *released_at*, *cancelled_at*, *is_late_cancellation*) con el valor expirada introducido en la Entrega 4. Quedan así cinco estados, tres de ellos terminales:

Estado	Tipo	Columnas que se pueblan al entrar	Significado en el dominio
reservada	inicial y activo	<i>confirm_deadline</i> (= <i>created_at</i> + 20 min)	El colaborador apartó el espacio para ese día; aún no confirma su llegada física.
ocupada	activo	<i>confirmed_at</i>	El colaborador confirmó que ocupó el espacio (botón "Ocupar parqueo").
liberada	terminal	<i>released_at</i>	El colaborador salió y liberó el espacio (salida normal o liberación anticipada voluntaria).
expirada	terminal	(sin marca propia; queda el <i>confirm_deadline</i> vencido)	El colaborador no confirmó en 20 minutos; el sistema liberó el espacio automáticamente.
cancelada	terminal	<i>cancelled_at</i> <i>is_late_cancellation</i>	El colaborador canceló la reserva antes de ocuparla.

Las transiciones, su disparador y quién las ejecuta

Cada transición tiene un disparador claro y un responsable de cómputo distinto. La siguiente tabla es la referencia central del componente: define qué actor inicia cada cambio de estado, qué servicio lo ejecuta y qué mecanismo garantiza la correctitud.

	Transición	Disparador	Ejecuta	Mecanismo
1	sin reserva → reservada	POST /reservar	API en ECS Fargate	INSERT protegido por índice único parcial, garantiza atomicidad.
2	reservada → ocupada	PATCH /reservas/:id/ocupar	API en ECS Fargate	UPDATE condicionado.
3	ocupada → liberada	PATCH /reservas/:id/liberar	API en ECS Fargate	UPDATE condicionado, libera el espacio en tiempo real.
4	reservada → expirada	EventBridge Scheduler → SQS → worker Lambda	Worker de liberación (Lambda)	UPDATE SET status='expirada' WHERE status='reservada' AND confirm_deadline < :scan_before.
5	reservada → cancelada	DELETE /reservas/:id	API en ECS Fargate	UPDATE condicionado, si ocurre dentro de la ventana de cancelación tardía marca is_late_cancellation.

Antes de evaluar el estado de origen, cada transición iniciada por el usuario (confirmar, liberar, cancelar) verifica que quien la solicita sea el dueño de la reserva; si no lo es, responde 403 sin importar el estado. La autorización por propiedad antecede a la validación de estado.

Tres observaciones de diseño que sostienen la coherencia de la máquina:

1. **Toda transición es un UPDATE condicionado al estado de origen.** Ninguna transición asume el estado actual: cada una lleva en su cláusula WHERE el estado del que parte. Esto las hace seguras frente a concurrencia y frente a reprocesamiento. Es el mismo patrón que vuelve idempotente al worker de liberación (Entrega 4), si dos peticiones intentan confirmar la misma reserva, solo la primera encuentra status = 'reservada'; la segunda no afecta filas.
2. **Solo la transición 1 necesita protección anti-concurrencia especial.** Las transiciones 2–5 operan sobre una reserva que ya existe y cuyo dueño es conocido, así que el WHERE por estado basta. La transición 1 es la única donde dos actores distintos compiten por el mismo recurso (el espacio-día), y por eso es la que descansa en el índice único parcial. Esta es la razón por la que la atomicidad se resolvió en la base de datos y no con colas (decisión de Entrega 2, reafirmada en Entrega 4).
3. **El comprobante es un efecto lateral, no una transición.** Al entrar a *reservada*, el API encola GenerateReceiptCommand y responde 201 sin esperar. El

worker genera el PDF, lo sube a S3 y publica ReceiptReadyEvent. La reserva no cambia de estado por esto; lo único que se actualiza es el puntero receipt_s3_key. Modelar el comprobante como efecto lateral asíncrono, y no como un estado de la reservam es lo que permite cumplir el SLA de "comprobante por correo en menos de 30 segundos" sin bloquear la confirmación de la reserva.

Dependencias del componente

La máquina de estados no vive aislada; depende de varios componentes ya diseñados:

- **PostgreSQL (RDS):** Es el dueño de la verdad. Toda transición es una escritura aquí, y la atomicidad de la transición 1 depende de sus índices únicos parciales.
- **API en ECS Fargate:** Encargado de ejecutar las transiciones 1, 2, 3 y 5 (las disparadas por el usuario).
- **EventBridge Scheduler + SQS + worker Lambda:** Estos ejecutan la transición 4 (la disparada por tiempo).
- **SQS + S3 + SNS:** Soportan el efecto lateral del comprobante al entrar a reservada.
- **Tabla settings:** Este parametriza la ventana de cancelación tardía que decide el flag is_late_cancellation en la transición 5.

API surface

Convenciones generales

El backend expone una API REST sobre HTTP, servida por el contenedor de la API en ECS Fargate detrás del Application Load Balancer (Entrega 2/Entrega 3). Todas las rutas aplican las siguientes convenciones:

- **Autenticación:** Todas las rutas requieren un JWT válido en el header Authorization, *Bearer <token>*, con dos únicas excepciones públicas: **POST /auth/login** (que emite el token) y **GET /health** (liveness probe). El token lo emite el servicio de autenticación externo (Cognito/Auth0, Entrega 2) y transporta system_role (driver / admin) y category del usuario.
- **Autorización en dos ejes:**
 - *system_role* decide qué endpoints puede invocar el usuario (un driver no accede a /admin/*).
 - *category* (ejecutivo / operativo / visitante_frecuente) decide qué espacios puede reservar, filtrando los resultados de disponibilidad y validando la reserva.
- **Formato:** El cuerpo de las peticiones y respuestas es JSON. Los nombres de campos en el contrato usan *snake_case* (space_id, reservation_date, confirm_deadline).
- **Tipos de vehículo:** El enum vigente es *auto*, *moto* y *camioneta*.
- **Códigos de error comunes:** Todos los endpoints comparten esta tabla de respuestas de error, por lo que en el detalle de cada endpoint solo se anotan los

códigos con semántica específica:

Código	Significado	Cuándo se produce
400	Validación	El payload o los query params no pasan el esquema de validación (tipo inválido, campo faltante).
401	No autenticado	Token ausente, inválido o expirado.
403	No autorizado	El rol o una regla de negocio prohíbe la acción (categoría sin acceso, no es el dueño de la reserva, usuario bloqueado).
404	No encontrado	El recurso (reserva, espacio, vehículo) no existe.
409	Conflicto	Choque contra el índice único parcial de PostgreSQL (error 23505): double-booking o segunda reserva activa del día.
422	Regla de negocio	La petición es válida pero viola una invariante (estado incorrecto para la transición, vehículo no aprobado, tipo de vehículo que no coincide con el espacio).

Endpoints por dominio

Autenticación y perfil

Método	Ruta	Rol	Descripción
POST	/auth/login	público	Inicia sesión. Recibe correo y contraseña; si son válidos devuelve un token JWT y los datos básicos del usuario (id, nombre, system_role, category). Cuenta inactiva o bloqueada, o credenciales incorrectas, devuelven 401.
GET	/auth/me	autenticado	Devuelve el perfil del usuario dueño del token.
POST	/auth/change-password	autenticado	Cambia la contraseña del propio usuario. Verifica la contraseña actual antes de reemplazarla; responde 204 sin cuerpo si tiene éxito y 400 si la actual es incorrecta.

Vehículos del conductor

Método	Ruta	Rol	Descripción
GET	/me/vehicles	driver	Lista los vehículos del propio conductor con su placa (descifrada), tipo y si ya están aprobados. Se usa para saber con qué vehículo puede

			reservar.
POST	/me/vehicles	driver	Registra una placa nueva. Queda en estado no aprobado hasta que un admin la apruebe. Una placa ya registrada en el sistema devuelve 409.
PATCH	/me/vehicles/:id	driver	Edita una placa propia, pero solo mientras no esté aprobada; intentar editar una ya aprobada devuelve 403. Evita que un conductor altere una placa que el admin ya validó.
DELETE	/me/vehicles/:id	driver	Elimina una placa propia. Se bloquea con 403 si el vehículo tiene reservas activas, para no dejar reservas huérfanas. Responde 204 si se elimina.

Disponibilidad

Método	Ruta	Rol	Descripción
GET	/availability?tipo_vehiculo=&fecha=	autenticado	Es la consulta central del sistema: devuelve el estado de cada espacio para una fecha. El estado no se lee de una columna, se calcula a partir de la reserva activa del día. Filtra automáticamente por la category del conductor y excluye espacios inactivos o en blackout.

Reservas

Método	Ruta	Rol	Descripción
POST	/reservar	driver	Crea la reserva. Antes de insertar valida que el vehículo sea del usuario y esté aprobado, que el tipo de vehículo coincida con el espacio, que la categoría del conductor tenga acceso a ese espacio y que no haya blackout en la fecha.
GET	/me/reservations	driver	Devuelve las reservas del propio conductor (la activa y su historial), ordenadas por fecha. Alimenta la vista de "mis reservas".
GET	/reservations/:id	admin	Devuelve el detalle de una reserva, enriquecido con datos del espacio y del vehículo, y el puntero al comprobante.
POST	/reservations/:id/confirm	driver	Confirma la llegada física ("Ocupar parqueo").
POST	/reservations/:id/release	driver	Libera el espacio y lo devuelve a disponible en tiempo real para que otro conductor lo tome el mismo día. Solo procede desde ocupada.

POST	/reservations/:id /cancel	driver	Cancela una reserva aún no ocupada. Solo procede desde reservada.
GET	/reservations/:id /receipt	admin	Devuelve una URL prefirmada de S3 para descargar el comprobante en PDF con QR.

Todas las rutas /admin/* exigen system_role = admin; un conductor recibe 403.

Administración: usuarios y aprobación de vehículos

Método	Ruta	Descripción
GET	/admin/users	Lista los colaboradores, con filtros opcionales por estado activo, rol y categoría. Es la base del gestor de usuarios.
POST	/admin/users	Crea un colaborador con su rol y categoría. El teléfono se guarda cifrado. Un correo ya registrado devuelve 409. Responde 201.
PATCH	/admin/users/:id	Edita nombre, rol, categoría o teléfono de un usuario. Incluye una salvaguarda: ningún admin puede cambiar su propio rol (403), evitando que se quiten privilegios por error.
PATCH	/admin/users/:id/deactivate /admin/users/:id/activate	Activa o desactiva el acceso de un usuario. Un usuario desactivado pierde el acceso pero conserva su historial.
GET	/admin/vehicles?approved=false	Lista las placas pendientes de aprobación, con los datos del dueño. Es la bandeja de aprobaciones del admin.
PATCH	/admin/vehicles/:id/approve	Aprueba una placa, habilitándola para reservar. Si ya estaba aprobada, la operación es idempotente.

Administración: espacios y blackouts

Método	Ruta	Descripción
GET	/admin/spaces	Devuelve el inventario completo de espacios (incluidos los inactivos), cada uno con sus categorías permitidas.
POST	/admin/spaces	Crea un espacio con su etiqueta, tipo de vehículo y las categorías de colaborador que pueden usarlo. Crea el espacio y sus categorías permitidas en una transacción. Responde 201.
PATCH	/admin/spaces/:id	Edita etiqueta, tipo o el conjunto de categorías permitidas de un espacio.

PATCH	/admin/spaces/:id /deactivate /admin/spaces/:id /activate	Activa o desactiva un espacio. Un espacio inactivo desaparece de la consulta de disponibilidad sin borrarse.
GET POST	/admin/spaces/:id /blackouts	Lista o crea inhabilitaciones de un espacio por rango de fechas. Valida que la fecha final no sea anterior a la inicial (400).
DELETE	/admin/blackouts/:id	Elimina un blackout, devolviendo el espacio a su disponibilidad normal. Responde 204.

Administración — dashboard, tarifas, historial y settings

Método	Ruta	Descripción
GET	/admin/dashboard/occupancy	Devuelve los conteos de espacios por estado y tipo de vehículo para el día actual. Es la fuente del dashboard del admin, que se refresca cada ≤ 30 s.
GET	/tariffs	Devuelve el precio vigente por tipo de vehículo. Accesible a cualquier usuario autenticado.
GET POST	/admin/tariffs	GET devuelve el historial completo de precios. POST registra un precio nuevo. El precio se guarda en GTQ por defecto.
GET	/admin/reservations?user_id=&from=&to=&status=	Devuelve el historial de reservas con filtros por usuario, rango de fechas y estado, enriquecido con datos de usuario y espacio. Es la vista de asistencia inmutable.
GET	/admin/reservations/export	Exporta ese mismo historial como archivo CSV descargable, con los mismos filtros.
GET PUT	/admin/settings /admin/settings/:key	GET lista la configuración. PUT crea o actualiza una clave.

Operación e internos

Método	Ruta	Descripción
--------	------	-------------

GET	/health	Liveness probe sin autenticación. Devuelve {status: "ok"} sin tocar la base de datos.
GET	/ready	Readiness probe: ejecuta una consulta trivial contra la base de datos. Devuelve 200 si la BD responde, 503 si no. Permite al orquestador saber si la tarea puede recibir tráfico.
	Trigger releaseExpired	No es un endpoint HTTP, es un worker. Ejecuta la transición reservada → expirada sobre las reservas no confirmadas tras 20 min

Estrategia de escalado y throttling

Escalado del cómputo

La API corre en ECS Fargate (Entrega 2). La carga del sistema es marcadamente pico: las reservas se concentran en la ventana 06:00–09:00, mientras que el resto del día domina el polling de disponibilidad. La estrategia es una política de Auto Scaling basada en utilización de CPU del servicio ECS, con un mínimo de una tarea siempre activa para eliminar el cold start de Fargate (los 20–30 s de arranque que la Entrega 2 reconoció como desventaja). Durante el pico la política añade tareas; fuera del pico vuelve al mínimo de una. No se escala a cero, decisión ya justificada en la Entrega 2. Las sondas **/health** y **/ready** permiten al orquestador decidir cuándo una tarea nueva está lista para recibir tráfico.

Throttling actual

El backend ya aplica un límite de tasa global con express-rate-limit: 120 peticiones por minuto por cliente, con cabeceras estándar de rate limit y respuesta 429 Too Many Requests al superarlo. Este límite se monta después de las sondas de salud, de modo que los health checks de ECS nunca se ven afectados. Es una protección sencilla y suficiente para el MVP.

Throttling objetivo

Para producción, este límite único y global se refinaría en dos niveles: límites por IP en el borde (Application Load Balancer) para frenar abuso antes de llegar al cómputo, y límites diferenciados por usuario autenticado (extraído del JWT) más estrictos sobre la ruta crítica de escritura **POST /reservar**, que es la de mayor contención, dejando más holgura al polling de lectura de **/availability**.

Protección de la operación crítica

Conviene subrayar que el throttling no es el mecanismo que previene el double-booking. Aunque dos peticiones legítimas pasen el límite de tasa y lleguen al mismo tiempo, la atomicidad la sigue garantizando el índice único parcial en PostgreSQL, que deja pasar solo una y devuelve 409 a la otra (Entrega 2). El throttling protege contra volumen abusivo; la integridad de la reserva la protege la base de datos.

Red detallada

Para proteger el sistema, aplicamos una segmentación estricta utilizando **Security Groups (SG)** que actúan como firewalls virtuales a nivel de recurso. Ningún componente tiene acceso libre; todo puerto y origen está explícitamente limitado mediante el principio de mínimo privilegio.

1.1. Arquitectura de Security Groups por Capa

La red se divide en tres capas principales distribuidas en dos Zonas de Disponibilidad (Multi-AZ) para garantizar alta disponibilidad:

1. **Capa del Balanceador (Subredes Públicas):** Aloja el Application Load Balancer (ALB).
2. **Capa de Cómputo (Subredes Privadas):** Aloja los contenedores de la aplicación en ECS Fargate.
3. **Capa de Datos y Servicios (Subredes Aisladas/Privadas):** Aloja la base de datos relacional (Amazon RDS PostgreSQL) y los VPC Endpoints.

Tablas : Reglas de Inbound y Outbound de los Security Groups

1. SECURITY GROUP: **sg-application-alb** (Asociado al Application Load Balancer)

- **REGLAS DE ENTRADA (INBOUND)**

Puerto	Protocolo	Origen	Razón/Justificación
80	TCP	0.0.0.0/0	Permite tráfico HTTP público para redirigir inmediatamente a HTTPS (puerto 443).
443	TCP	0.0.0.0/0	Permite tráfico HTTPS seguro desde internet para que conductores y administradores accedan.

- **REGLAS DE SALIDA (OUTBOUND)**

Puerto	Protocolo	Destino	Razón/Justificación
8080	TCP	sg-ecs-fargate	Envía el tráfico limpio y balanceado únicamente a los contenedores de Express.js.

2. SECURITY GROUP: **sg-ecs-fargate** (Asociado a las Tareas de ECS Fargate)

- **REGLAS DE ENTRADA (INBOUND)**

Puerto	Protocolo	Origen	Razón/Justificación
8080	TCP	sg-application-alb	Solo acepta peticiones HTTP que provengan del balanceador. Nadie de internet entra directo.

- **REGLAS DE SALIDA (OUTBOUND)**

Puerto	Protocolo	Destino	Razón/Justificación
5432	TCP	sg-rds-postgresql	Permite a la app conectarse a PostgreSQL para validar disponibilidad y registrar reservas.
443	TCP	sg-parking-vpce (nuevo)	Permite conectarse a los servicios de AWS (SQS, S3, Secrets Manager) de forma interna.

3. SECURITY GROUP: **sg-rds-postgresql** (Asociado a la Base de Datos Amazon RDS)

- **REGLAS DE ENTRADA (INBOUND)**

Puerto	Protocolo	Origen	Razón/Justificación
5432	TCP	sg-ecs-fargate	Permite a Fargate (API) conectarse a PostgreSQL
5432	TCP	sg-parking-lambda	Permite a las Lambdas (receipt, email, release, rotación) conectarse a PostgreSQL

- **REGLAS DE SALIDA (OUTBOUND)**

Puerto	Protocolo	Destino	Razón/Justificación
--------	-----------	---------	---------------------

Todo	Todo	Ninguno (o)	La base de datos no necesita iniciar conexiones hacia ningún otro destino.
------	------	-------------	--

4. SECURITY GROUP: **sg-parking-vpce** (Asociado a los VPC Endpoints de AWS)

- **REGLAS DE ENTRADA (INBOUND)**

Puerto	Protocolo	Origen	Razón/Justificación
443	TCP	sg-ecs-fargate	Permite que Fargate interactúe de forma segura con el API de SQS, Secrets Manager y S3.
443	TCP	sg-parking-lambda	Permite a las Lambdas (receipt, email, release, rotación) conectarse a PostgreSQL

- **REGLAS DE SALIDA (OUTBOUND)**

Puerto	Protocolo	Destino	Razón/Justificación
Todo	Todo	Ninguno (o)	Los interfaces de endpoints solo reciben peticiones, no inician tráfico saliente.

5. SECURITY GROUP: **sg-parking-lambda**

- **REGLAS DE SALIDA (OUTBOUND)**

Puerto	Protocolo	Destino	Razón/Justificación
5432	TCP	sg-rds-postgresql	Lambdas leen/escriben datos y ejecutan rotación de secretos
443	TCP	sg-parking-vpce	Para que la lambda receipt-worker se conecte en SNS_EP y SQS_EP(ambos endpoints).

Nota de optimización de costos y redes: Para interactuar de forma segura con Amazon S3, Amazon SQS y AWS Secrets Manager sin incurrir en los altos costos de transferencia por hora de un NAT Gateway, se implementan **VPC Endpoints (AWS PrivateLink)** de tipo Interface y Gateway dentro de la red privada. Esto asegura que el tráfico jamás salga a la red pública de internet.

1.2. Flujo de Tráfico End-to-End

A continuación, se detalla mecánicamente la trayectoria que realiza el tráfico en los dos escenarios críticos del negocio:

Caso de Uso A: El conductor crea una reserva desde la aplicación móvil/web

1. **Petición del Cliente:** El usuario presiona "Confirmar Reserva" en el frontend (Next.js). El navegador web emite una solicitud HTTPS al dominio del sistema (ejemplo: `api.parking-company.com`) en el puerto 443.
2. **Recepción en la Capa Pública:** El tráfico llega a través de Internet Gateway al Application Load Balancer (ALB) ubicado en las subredes públicas. El ALB evalúa la regla de su grupo de seguridad (`sg-application-alb`), descripta el certificado SSL/TLS y valida la petición.
3. **Enrutamiento Interno:** El ALB redirige la petición HTTP limpia a través de las subredes privadas hacia una de las tareas activas de **ECS Fargate** en el puerto 8080. El tráfico es aceptado porque el grupo de seguridad de Fargate (`sg-ecs-fargate`) permite explícitamente el ingreso desde `sg-application-alb`.
4. **Lógica de Negocio y Persistencia:** El backend (Express.js) procesa la solicitud. Para guardar la reserva, abre una conexión TCP en el puerto 5432 contra la instancia de Amazon RDS PostgreSQL. El grupo de seguridad `sg-rds-postgresql` le da acceso porque el origen es el identificador de `sg-ecs-fargate`.
5. **Generación de Comprobantes:** Una vez confirmada la base de datos, el backend genera un comprobante digital de parqueo en PDF y lo sube de forma asíncrona a un Bucket de Amazon S3 utilizando el VPC Endpoint correspondiente. El tráfico fluye directo por la red interna de AWS sin tocar internet.

Caso de Uso B: Worker de liberación automática de espacios expirados (Trigger releaseExpired)

1. **Disparo Temporal (EventBridge):** Al cumplirse la ventana de inactividad de 20 minutos de una reserva no confirmada(dicha actividad se empieza a contar después del horario origen estipulado por el conductor), un evento interno de AWS dispara un mensaje hacia la cola Amazon SQS (`release-queue`).
2. **Consumo asíncrono:** Las funciones serverless AWS Lambda encargadas del procesamiento de mensajería asíncrona (`lambda_release`) realizan la petición de sondeo (polling) de manera privada a través de las interfaces locales del VPC

Endpoint de SQS utilizando el puerto seguro 443 controlado por el Security Group **sg-parking-vpce**.

3. **Ejecución del Cambio de Estado:** Tras consumir el mensaje de expiración de la cola, el worker ejecuta la consulta de actualización **reserved** → **expired** directamente en la base de datos RDS a través del puerto 5432, liberando de inmediato el inventario de espacios para que otros colaboradores lo utilicen.

Modelo de seguridad

La seguridad en AWS se rige bajo la premisa de que "toda identidad de software debe tener permisos estrictos para operar". Se descartan por completo el uso de usuarios IAM con credenciales estáticas en producción y la política permisiva **AdministratorAccess**.

2.1. Roles IAM por Servicio (Mínimo Privilegio)

Definimos dos roles IAM específicos e independientes para aislar las responsabilidades operativas dentro de ECS Fargate:

a. ECS Task Execution Role (**role-parking-execution**)

Este rol lo utiliza la plataforma de AWS (el agente de ECS) para levantar y preparar el contenedor antes de arrancar el código de la aplicación.

- **Acciones Específicas Permitidas:**
 - **ecr:GetAuthorizationToken:** Requerido para autenticarse contra el registro de contenedores de AWS.
 - **ecr:BatchCheckLayerAvailability,**
ecr:GetDownloadUrlForLayer: Permite descargar la imagen de Docker de la aplicación (Next.js/Express.js) desde Amazon ECR.
 - **logs:CreateLogStream, logs:PutLogEvents:** Permite enviar los logs del contenedor de la aplicación directamente a Amazon CloudWatch para auditoría.
 - **secretsmanager:GetSecretValue:** Permite extraer de forma segura las credenciales de la base de datos al inicializar.
- **Recursos Específicos:**
 - Contenedor:
arn:aws:ecr:us-east-1:123456789012:repository/repo-parking-app donde el nombre del repositorio real es "cicd-foundation-project".
 - Secreto:
arn:aws:secretsmanager:us-east-1:123456789012:secret:parking/prod/rds**-<XYZ>**
 - Logs:
arn:aws:logs:us-east-1:123456789012:log-group:/aws/ecs/parking-prod**:***

b . ECS Task Role (role-parking-application)

Este rol es el que asume directamente nuestro código de node.js/express.js una vez que ya está corriendo. Determina qué puede hacer nuestra aplicación dentro de la infraestructura de AWS.

- **Acciones Específicas Permitidas:**

- **s3:PutObject, s3:GetObject:** Requerido exclusivamente para escribir y leer los PDFs de los comprobantes digitales de parqueo en S3. No se otorga **s3:DeleteObject** para evitar pérdidas maliciosas o accidentales de datos.
- **sqs:SendMessage.**
- **kms:Decrypt, kms:GenerateDataKey:** Permite descifrar y cifrar los datos del negocio que están resguardados con llaves maestras.
- **dynamodb:PutItem, dynamodb:GetItem, dynamodb:Query** limitado al ARN de la tabla de auditoría.

- **Recursos Específicos:**

- Bucket S3: **arn:aws:s3:::bucket-parking-receipts/***
- Cola SQS:
arn:aws:sqs:us-east-1:123456789012:receipt-queue
- Llave KMS:
arn:aws:kms:us-east-1:123456789012:key/parking-prod-key

c. AWS Lambda Role (role-lambda-receipt-worker)

- **Acciones Específicas Permitidas:**

- **sqs:ReceiveMessage, sqs:DeleteMessage, sqs:GetQueueAttributes, s3:PutObject, s3:GetObject, sns:Publish, secretsmanager:GetSecretValue, kms:Decrypt, kms:GenerateDataKey, logs:CreateLogStream, logs:PutLogEvents.**

- **Recursos Específicos:**

- Cola **receipt-queue**, bucket de comprobantes, tópico **receipt-ready-topic**, secreto para credenciales de RDS, llaves KMS y CloudWatch Logs.

d. AWS Lambda Role (role-lambda-release-worker)

- **Acciones Específicas Permitidas:**

- **sqs:ReceiveMessage, sqs:DeleteMessage, sqs:GetQueueAttributes, secretsmanager:GetSecretValue,** acciones de logs y KMS.

- **Recursos Específicos:**

- Cola **release-queue**, secreto para credenciales de RDS, CloudWatch Logs y llaves KMS.

c. AWS Lambda Role (`role-lambda-email-worker`)

- **Acciones Específicas Permitidas:**
 - `sqs:ReceiveMessage`, `sqs>DeleteMessage`, `sqs:GetQueueAttributes`, `secretsmanager:GetSecretValue`, acciones de logs y KMS.
- **Recursos Específicos:**
 - Cola `receipt-email-queue`, secreto para credenciales de RDS, CloudWatch Logs y llaves KMS.

2.2. Gestión de Secretos

Toda credencial confidencial se extrae del código fuente y de las variables de entorno tradicionales. En su lugar se inyectan en tiempo de ejecución utilizando AWS Secrets Manager.

● Tabla 2.2: Matriz de Control de Secretos del Sistema

Nombre del Secreto	Dueño / Propietario	Estrategia de Rotación Automática
<code>parking/prod/rds-credentials</code>	Equipo DBA / DevOps	Cada 30 días mediante una función Lambda nativa de AWS que actualiza la contraseña en PostgreSQL y actualiza el valor del secreto sin tirar la app.
<code>parking/prod/jwt-session-key</code>	Equipo de Desarrollo	Cada 90 días de forma manual/programada en ventanas de bajo tráfico para invalidar sesiones antiguas.

En otras palabras, cuando la tarea de ECS Fargate arranca, lee el secreto del Secrets Manager. Si la contraseña de la base de datos cambia automáticamente cada 30 días, la aplicación vuelve a consultar el almacenamiento de secretos para obtener la nueva clave válida sin que el equipo de ingeniería intervenga manualmente o deje registros en texto plano.

Nota sobre la lambda: A la Lambda de rotación automática de Secrets Manager, aunque es administrada por AWS, ejecuta dentro de tu VPC y utiliza el mismo `sg-parking-lambda` que las otras funciones Lambda del sistema, lo que simplifica la arquitectura sin sacrificar seguridad." En otras palabras, no es un componente nuevo, es uno ya integrado en AWS SM.

2.3. Llaves KMS y su Alcance

Para centralizar el control del cifrado del sistema, se implementa una **Llave Administrada por el Cliente (CMK - Customer Managed Key)** en *AWS KMS*, en lugar de utilizar las llaves genéricas por defecto de AWS. Esto nos otorga el control absoluto de auditar quién descifra información a través de *AWS CloudTrail*.

- **Nombre/Alias de la Llave:** `alias/parking-prod-key`
- **Alcance Técnico:**
 - **Cifrado de Base de Datos (RDS):** Protege el almacenamiento físico en bloques de la instancia PostgreSQL, incluyendo las tablas de usuarios, parqueos y bitácoras de reservas.
 - **Cifrado de Almacenamiento de Objetos (S3):** Se asocia al bucket de comprobantes digitales. Cada archivo cargado se encripta de forma transparente con esta llave bajo el estándar AES-256.
 - **Cifrado de Mensajería (SQS):** Encripta los payloads de los mensajes en tránsito dentro de la cola de reservas expiradas.
- **Política de la Llave (Key Policy):** Restringe el uso de la función `kms:Decrypt` únicamente si la petición proviene explícitamente del ARN del rol de la aplicación (`role-parking-application`). Si un administrador de la nube intenta descargar un PDF directamente de S3 sin este rol, el servicio denegará la lectura del archivo.

2.4. Cifrado en Tránsito y en Reposo

Garantizamos una estrategia de protección de datos de punta a punta cubriendo los dos estados obligatorios:

Cifrado en Tránsito (Data in Transit)

- **Externo:** Toda comunicación iniciada por los conductores o administradores viaja protegida sobre la red pública mediante el protocolo **TLS 1.2 o superior** de forma estricta. El Application Load Balancer valida el certificado SSL provisto y renovado automáticamente por *AWS Certificate Manager (ACM)*.
- **Interno:** La comunicación desde el contenedor en ECS Fargate hacia la base de datos PostgreSQL viaja forzada con parámetros de conexión segura (**SSL Mode: require**), evitando ataques de interceptación de datos (*Man-in-the-Middle*) dentro de la propia red interna.

Cifrado en Reposo (Data at Rest)

- **Persistencia de Datos:** El almacenamiento físico de Amazon RDS y los discos efímeros adjuntos a las tareas de ECS Fargate están cifrados utilizando criptografía de hardware **XTS-AES-256** mediante nuestra llave KMS.
- **Archivos Históricos:** El bucket de S3 tiene la directiva estricta de denegar cualquier carga de archivos que no incluya los encabezados de cifrado del lado del servidor (**Server-Side Encryption with AWS KMS keys - SSE-KMS**).

Plan de observabilidad

El sistema actualmente cuenta con una capa de logging estructurado basada en Pino (v9) y pino-http (v10). Cada petición HTTP genera una línea JSON que incluye el método, la ruta, el código de respuesta y el tiempo de respuesta. Los errores no controlados se registran con `logger.error({ err })` en el middleware centralizado (`errorHandler.ts`). En producción el nivel de log es `info` ; en otros entornos, `debug` . Este output va a `stdout` del contenedor, que el agente de ECS debe redirigir a un log group de CloudWatch Logs mediante el driver `awslogs` . Sin embargo, existen brechas importantes que impiden operar el sistema con visibilidad adecuada. La más crítica es la ausencia de un identificador de correlación por petición: actualmente no es posible agrupar todas las líneas de log que pertenecen a una misma solicitud HTTP, lo que dificulta enormemente el diagnóstico de errores en producción. Tampoco se adjunta el identificador del usuario autenticado a las líneas de log, lo que impide determinar qué usuario desencadenó un error sin decodificar manualmente el token JWT. Finalmente, no existe ningún mecanismo de métricas, trazado distribuido ni dashboards.

1.1 Logging y Correlación de Peticiones

El plan para cerrar la brecha de correlación es agregar un campo `requestId` único a cada petición HTTP. El `requestId` se genera en la capa de middleware de logging (`pino-http`) al inicio de cada solicitud usando `crypto.randomUUID()`. Todos los logs emitidos durante el ciclo de vida de esa petición heredan el mismo valor gracias al logger hijo que `pino-http` crea por request. Adicionalmente, una vez que el middleware `requireAuth` verifica el token JWT, el `userId` se añade al mismo logger hijo, de modo que todos los logs posteriores en la cadena de middlewares incluya también el identificador del usuario.

El formato objetivo para las líneas de log en producción es el siguiente:

Campo	Descripción	Ejemplo
level	Nivel de log	2026-06-13T08:05:00.000Z
time	Timestamp UTC	
requestId	Id unico por request	req_a1b2c3d4e5f6
userId	id de usuario	42
method	Metodo http	POST
url	Ruta de la petición	/reservar
status	Código de respuesta	201
responseTime	Duración de la respuesta	48
msg	Mensaje descriptivo	request completed

1.2 Métricas RED por Endpoint Principal

Las métricas RED (Rate, Errors, Duration) describen el estado de salud de un servicio desde la perspectiva del usuario. Para este sistema, las fuentes primarias son CloudWatch Logs Insights (sobre los logs de pino-http) y las métricas nativas de los servicios AWS (ALB, RDS, SQS, ECS).

Métrica	Unidad	Fuente	Endpoint
Solicitudes totales por minuto	count/min	ALB requestCount	API, todos los endpoints
Reservas creadas por minuto	count/min	Metric filter de status 201	POST /reservar
Consultas de disponibilidad por minuto	count/min	Insighths	GET /availability
Errores http 5xx	count/min	ALB HTTPCode_Target_5XX_Count	API - todos los endpoints
fallos de generación de comprobante	count/min	Metric filter sobre “receipt generation failed”	POST /reservar - bloque s3 async
Latencia de disponibilidad	ms	logs Insight sobre campo reponseTime	GET /availability
Latencia de reserva	ms	logs Insights campo responseTime	POST /reservar
Latencia de escritura en RDS	count/min	CloudWatch RDS WriteLatency	PostgreSQL 16

1.1 Estrategia de Alertas

Se definen cuatro alertas mínimas para operar el sistema en producción. Todas se implementan en CloudWatch Alarms y notifican al equipo de ingeniería vía SNS.

Alerta 1 — Tasa de errores 5xx elevada

Métrica: ALB HTTPCode_Target_5XX_Count en el target group del API.

Umbral: Más de 5 errores en una ventana de 5 minutos.

Ventana de evaluación: 5 minutos, evaluada cada minuto.

Severidad: Alta.

Impacto de negocio: Cualquier respuesta 5xx durante el horario de reservas matutinas (7–9 AM) bloquea a los colaboradores antes de llegar al trabajo. El sistema es

sensible al tiempo: 15 minutos de errores en ese tramo horario inutilizan la plataforma para toda la jornada.

Alerta 2 — Mensajes en Dead-Letter Queue

Métrica: CloudWatch SQS ApproximateNumberOfMessagesVisible en cualquiera de los tres DLQs (receipt-dlq , release-dlq , email-dlq).

Umbral: Cualquier valor > 0.

Ventana de evaluación: 5 minutos.

Severidad: Crítica para release-dlq ; Alta para las demás.

Impacto de negocio: Un mensaje en release-dlq significa que la liberación automática de una reserva expirada falló 3 veces consecutivas. El espacio queda bloqueado como "Reservado" para el resto del día aunque nadie lo ocupe, reduciendo la capacidad efectiva del parqueo.

Alerta 3 — Espacio libre en RDS por debajo del umbral

Métrica: CloudWatch RDS FreeStorageSpace de la instancia PostgreSQL.

Umbral: Menos de 3 GB (15% de los 20 GB asignados).

Ventana de evaluación: 15 minutos, evaluada cada 5 minutos. Severidad: Crítica.

Impacto de negocio: Cuando el almacenamiento de RDS se agota, cada INSERT en reservations , tariffs y demás tablas falla con el código PG 53100 (disk_full). Toda la ruta de escritura queda inutilizada: sin nuevas reservas, sin confirmaciones, sin liberaciones.

Alerta 4 — Número de tareas ECS del API en cero

Métrica: CloudWatch ECS RunningTaskCount para el servicio api-service . Umbral: Menos de 1 tarea en ejecución durante más de 2 minutos consecutivos. Ventana de evaluación: 2 minutos, evaluada cada minuto.

Severidad: Crítica.

Impacto de negocio: Con cero tareas activas, los health checks del ALB fallan y el balanceador retorna 503 para todas las solicitudes. Toda la aplicación queda inaccesible sin ninguna ruta de degradación alternativa.

Estimado de costo

El proveedor de nube es **AWS en la región us-east-1**, que es donde se despliegan todos los recursos definidos en el módulo. La herramienta de observabilidad propuesta es **CloudWatch nativo** (Logs, Metrics, Alarms, Dashboards), que es el stack más económico y sin fricción para servicios ya alojados en AWS.

Los supuestos de escala se basan en la naturaleza del sistema: una herramienta interna corporativa de una sola empresa, usada principalmente de 7:00 a 9:30 AM en días laborales, con un volumen estimado de 50 a 200 usuarios activos diarios. Cada petición HTTP genera aproximadamente 1 a 2 KB de log estructurado (JSON de Pino). Los precios son los publicados por AWS a junio de 2026 para us-east-1 y se expresan en orden de magnitud.

Los tres escenarios de logs quedan dentro del tier gratuito de CloudWatch Logs (5 GB/mes de ingesta). El volumen de almacenamiento activo con retención de 30 días no supera los 3.6 GB en el escenario alto, lo que genera un costo de almacenamiento inferior a 0.11 USD/mes.

Servicio / Ítem	Escenario Bajo	Escenario Esperado	Escenario Alto
Ingesta de logs (5 GB gratis)	USD 0.00	USD 0.00	USD 0.00
Almacenamiento de logs (30 días)	< USD 0.01	USD 0.01	USD 0.02
CloudWatch Metrics (~70 de pago × USD 0.30)	USD 21.00	USD 21.00	USD 21.00
Métricas personalizadas (metric filters, ~5 × USD 0.30)	USD 1.50	USD 1.50	USD 1.50
CloudWatch Alarms (4 alarmas × USD 0.10)	USD 0.40	USD 0.40	USD 0.40
CloudWatch Dashboard (1 × USD 3.00)	USD 3.00	USD 3.00	USD 3.00
Logs Insights queries	USD 0.10	USD 0.25	USD 1.00
Total mensual estimado	~USD 26	~USD 26	~USD 27

Riesgos y decisiones pendientes

Riesgo	Crítica	Descripción
Definición de tarea ECS sin logConfiguration	Crítica	El módulo infra/modules/compute/ no está presente en el repositorio (solo existe modules/.gitkeep). Si la task definition no configura el driver awslogs apuntando a un log group de CloudWatch, todo el output de Pino se descarta silenciosamente. Ninguna otra capa de observabilidad puede funcionar sin este vínculo.
Sin timeout de conexión ni de consulta en el pool de PostgreSQL	Crítica	New Pool({ connectionString }) en backend/src/db/index.ts:8 usa los valores predeterminados de node-postgres , que no tienen timeout de conexión ni de sentencia SQL. Una consulta lenta o una pérdida de conectividad con RDS bloquea el proceso de Node.js indefinidamente sin emitir ningún log ni métrica de error.
Worker de cron en-proceso en múltiples tareas ECS	Alta	El releaseExpiredWorker (backend/src/workers/releaseExpired.ts:11) corre cada minuto dentro del proceso del API. Si ECS lanza más de una tarea del servicio api-service , cada instancia ejecuta el mismo UPDATE concurrentemente. El índice único parcial de reservations evita inconsistencias de datos, pero genera escrituras redundantes en RDS cada minuto.
Generación de comprobante PDF sin ruta de reintento	Media	La subida a S3 dentro de POST /reservar (reservations.routes.ts:104) es un bloque void async fire-and-forget. Si S3 no está disponible o lanza un error, el campo receipt_s3_key en la base de datos queda como NULL . Solo se emite un log de error, sin alerta, sin reintento, y sin mecanismo para que el administrador reprocese los comprobantes faltantes.
Sin Multi-AZ en RDS en la configuración de dev	Media	El archivo infra/variables.tf define multi_az = false como valor por defecto. Si este valor se usa en producción, un fallo de zona de disponibilidad en us-east-1a implica una interrupción de aproximadamente

		30 minutos hasta que RDS reinicia en otra AZ.
--	--	--

Anexo IA

1. Observabilidad del sistema

a. ¿Dónde fue útil la IA?

Se necesitaba documentar el estado de observabilidad del sistema de parqueos para identificar qué tan visible era el comportamiento del sistema en producción y qué faltaba implementar. Para abordar esto, se consideraron dos caminos: realizar el análisis manualmente revisando el código y la infraestructura archivo por archivo, o utilizar una herramienta de IA para acelerar la lectura del repositorio y estructurar las recomendaciones. Se optó por usar IA porque el sistema tiene múltiples capas (backend en Express, infraestructura en Terraform, colas SQS, RDS en AWS) y revisar todo manualmente habría tomado considerablemente más tiempo.

b. ¿Dónde no fué útil y qué sesgos presentó?

No fue muy útil en generar nombres, y componentes consistentes a las anteriores decisiones, lo cuál generaba doble revisión y chequeo mediante otros agentes más poderosos para mejor verificación.

c. ¿Qué se descartó tras la revisión humana?

La IA generó una primera propuesta muy extensa que incluía recomendaciones como RDS Proxy, caché con Redis y mecanismos de reintento avanzados, la cual fue rechazada por ser demasiado compleja para el alcance del proyecto.

d. Lecciones aprendidas y conclusiones de diseño

Se le indicó que simplificara el plan enfocándose únicamente en los endpoints principales del API y en las decisiones más urgentes para producción. Con esa dirección, se produjo un plan más conciso que fue revisado y ajustado en estructura para incorporarse al entregable final El cual fue descrito en el entregable 5.

2. Diseño del sistema

a. ¿Dónde fue útil la IA?

- **Velocidad de estructuración y formateo:** Cuando se trató del diseño del sistema, el ordenamiento de las ideas y generación de propuesta de infraestructura del sistema, nos dió una excelente base para empezar, mejorando más y más hasta obtener nuestro sistema completo y ajustado a nuestro criterio.
- **Compleitud de políticas de mínimo privilegio:** En la sección de Modelo de Seguridad en la Entrega 5 nos permitió en base a nuestra lógica de negocio generar las políticas necesarias para adaptarlas al sistema, los roles recomendados y principios de mínimo privilegio, con esto pudimos evitar errores humanos en olvidarnos de algún principio necesario al igual de verificar no romper esta misma.
- **Redacción explicativa:** Ayudó a traducir conceptos abstractos o puramente de código a descripciones en lenguaje sencillo aptas para la comprensión de un ingeniero de infraestructura ajeno al desarrollo inicial. Esto fué vital no

solo para la redacción en sí, sino para tener mejores palabras explicativas y sencillas para cualquier redactor junior, middle a senior.

b. ¿Dónde NO fue útil y qué sesgos presentó?

- **Generalización ciega de la industria:** El mayor desafío de la IA fue su tendencia a proponer "soluciones estándar de internet" en lugar de respetar las decisiones de diseño tomadas por el grupo en fases previas. Por ejemplo, solía generar respuestas que no estaban acorde a lo especificado a anteriores entregas, generando inconsistencias entre ellas.
- **Alucinación de Componentes:** Por ejemplo en esta Entrega 5 Intentó forzar la existencia de un componente "Worker" persistente dentro del clúster de ECS Fargate para el procesamiento asíncrono, ignorando que la arquitectura del grupo ya había delegado esa responsabilidad de forma eficiente a funciones AWS Lambda independientes. Entre ellas y muchos componentes(tanto nuestro proyecto escalaba en infraestructura) se solían mezclar con otros al igual que los nombres,
- **Relación de componentes faltantes(olvidadas):** Claro estaba que muchas veces olvidaba considerar o preguntar sobre ciertas implementaciones, lo que hacía que nos generaba fricción en componentes faltantes.

c. ¿Qué se descartó tras la revisión humana?

- **Configuraciones de red flotantes:** Se rechazó la premisa de asignar firewalls a SQS/SNS y se reorientó estrictamente hacia el aseguramiento del `sg-parqueos-vpce`.
- En general, en las primeras implementaciones de diseño base en nuestro sistema, muchas veces proponía servicios como Lambda o EC2 en nuestro diseño cuando lo más eficiente era AWS Fargate; entre estas y otras propuestas que a lo largo del tiempo fuimos cambiando.

d. Lecciones aprendidas y conclusiones de diseño

- **La IA es un acelerador, no un arquitecto:** Actúa como un excelente generador de plantillas y texto base, pero carece de memoria contextual absoluta sobre las restricciones acumuladas del proyecto. La supervisión de un ingeniero es indispensable.
- **Importancia de la Verificación Cruzada:** El uso de herramientas de IA obliga al equipo a realizar auditorías más minuciosas de sus propios diagramas (como el código Mermaid D4) para contrastar y defender las decisiones de diseño del grupo frente a las sugerencias automáticas.
- **Darle el mejor y conciso diseño posible:** Es decir, conforme un proyecto avanza y va escalando en complejidad de cantidad de componentes, el mejor resumen sobre todo lo hecho en el proyecto es clave para que la IA no solo tenga el recuerdo de sólo una "Entrega" como en nuestro caso. Entre ella y muchas técnicas fueron aplicadas como, entrega de instrucciones en ``md``,

instrucciones específicas y detalladas en otros archivos del mismo formato, no generando muchos chats(gastando tokens) sino que todo el mismo contexto e instrucciones de implementación sean respondidos en la menor cantidad de outputs de la IA posible, ser bien específicos, no saturando de info a la IA, adjuntarle solo el contenido más importante, entre otros.

3. API surface

a. ¿Dónde fue útil la IA?

- **Traducción de la lógica de rutas a descripciones legibles:** ayudó a convertir el código de cada endpoint (validaciones, condiciones, códigos de respuesta) en descripciones en lenguaje claro que explican qué hace cada ruta, qué valida y a qué caso de uso responde, aptas para un ingeniero ajeno al desarrollo inicial.
- **Mapeo de endpoints a casos de uso:** vinculó cada endpoint con los casos de uso y funcionalidades específicas de las entregas anteriores (CU1–CU8, Fn1–Fn5), manteniendo la coherencia del documento maestro.

b. ¿Dónde NO fue útil y qué sesgos presentó?

- **Tendencia a mejorar la nomenclatura sin respetar el código:** inicialmente la IA propuso renombrar POST /reservar a POST /reservations y usar el verbo PATCH para las transiciones, siguiendo convenciones REST "de manual". Esto contradecía las rutas reales del proyecto. El sesgo fue claro: optimizar hacia un estándar genérico de la industria en lugar de documentar lo que el equipo ya había construido.

c. ¿Qué se descartó tras la revisión humana?

- **El renombramiento de endpoints:** se rechazó la propuesta de POST /reservations y PATCH, y se conservaron las rutas exactas del código (POST /reservar, POST /reservations/:id/confirm|release|cancel).

d. Lecciones aprendidas y conclusiones de diseño

- **El código es la fuente de verdad:** cuando existe implementación, hay que verificar contra ella. Varias afirmaciones plausibles que la IA generó desde las entregas anteriores resultaron imprecisas frente al código real. La verificación cruzada contra los archivos de rutas fue indispensable.